

ENGINEERING HANDBOOK / DEVELOPMENT STANDARD v1.0

FreeRTOS 기반 차량 다중 ECU 네트워크 및 CAN-LIN Gateway 개발

7 인 팀 프로젝트 개발 표준안 및 실무 매뉴얼

문서의 목적

CAN 기반 제어 ECU 와 LIN 기반 Body ECU 를 FreeRTOS Gateway 로 연결하는 프로젝트의 설계, 구현, 검증, 협업 규칙을 한 문서에서 추적할 수 있도록 정리한다.

항목	내용
대상	7 인 팀 프로젝트 / 임베디드·차량 네트워크 실무형 개발
핵심 인터페이스	CAN 500 kbps / LIN 19.2 kbps / PC Diagnostic Tool via CANable
핵심 산출물	vehicle.dbc, LIN Matrix/Schedule, FreeRTOS Gateway Firmware, PC Tool, Test Evidence
기본안	실제 모터 없이 Virtual Motor ECU 로 시작하고, 실제 모터 연동은 범위 밖 또는 확장 목표로 둔다.
기준일	2026-09-02 / 초안은 M1 Interface Freeze 에서 확정한다.

본 매뉴얼은 내용 생략보다 추적성과 재현성을 우선한다. 예시 ID 와 시간은 팀 리뷰를 거쳐 확정한다.

READING MAP

목차

이 문서는 프로젝트를 처음 세팅하는 순서와 실제 구현·검증 순서를 함께 반영한다. 먼저 1-4 장에서 범위와 협업 구조를 고정하고, 5-12 장에서 네트워크와 펌웨어를 구현한 뒤, 13-16 장에서 일정·협업·출시 운영을 반복한다.

1. 프로젝트 개요와 최종 아키텍처	3
2. ECU 역할과 7인 역할 분담	6
3. 전체 Tool Stack 과 핵심 Tool Chain	9
4. Repository 와 산출물 관리	12
5. CAN 설계, CANdb++, DBC 검증	14
6. LIN Matrix 와 Schedule 설계	18
7. MCU Bring-up 과 UART Logging	20
8. Gateway FreeRTOS 구조와 Routing	22
9. ECU 애플리케이션 구현	25
10. Fault Handling 과 Fault Injection	27
11. PC Diagnostic Tool	29
12. Automated Test 와 KPI 측정	31
13. 12 주 Roadmap 과 Milestones	33
14. 단계별 개발계획	35
15. Jira + Git/GitHub 협업 매뉴얼	43
16. Sprint 예시와 최종 치트시트	51
A. 최종 Demo 시나리오와 체크리스트	56
최종 한 장 치트시트	54

사용 원칙

문서의 각 완료조건은 Jira Task 의 acceptance criteria 와 연결한다. 특히 Interface Freeze 이후의 CAN/LIN 변경은 Excel 초안, CANdb++/DBC, 코드, 테스트 증적을 모두 같은 Jira Issue 와 PR 에서 추적한다.

1. 프로젝트 개요와 최종 아키텍처

이 프로젝트는 차량 제어 영역을 CAN, 차체·바디 영역을 LIN 으로 나누고, FreeRTOS 기반 Gateway ECU 가 두 네트워크 사이의 Signal Routing, 주기 변환, 상태 감시, Fault Handling 을 수행하는 분산 전장 시스템을 만드는 7 인 팀 프로젝트다.

핵심 목표

단순한 CAN/LIN 송수신 데모가 아니라, 요구사항 -> Interface Definition -> Firmware -> PC Diagnostic -> Automated Test -> HW Evidence 까지 하나의 재현 가능한 개발 흐름을 완성한다.

1.1 목표와 성공 기준

- CAN 500 kbps 에서 VCU, Motor ECU, Gateway ECU, Cluster ECU 가 정해진 Message/Signal 을 주고받는다.
- LIN 19.2 kbps 에서 Gateway 가 LIN Master 로 Door ECU 와 Light ECU 를 Schedule 에 따라 Polling 한다.
- Gateway 가 CAN Signal 을 해석해 LIN Frame 으로 변환하고, LIN 상태를 CAN 상태 메시지로 변환한다.
- CAN timeout, LIN timeout, Alive Counter 오류, ECU offline 을 감지하고 Cluster 와 PC Tool 에 전달한다.
- PC Diagnostic Tool 에서 DBC decode, raw monitor, ECU health, Logging, Fault Injection, Test Automation 이 가능하다.
- 모든 기능은 Jira Issue, Git Branch, Commit, PR, HW Test Evidence 로 추적된다.

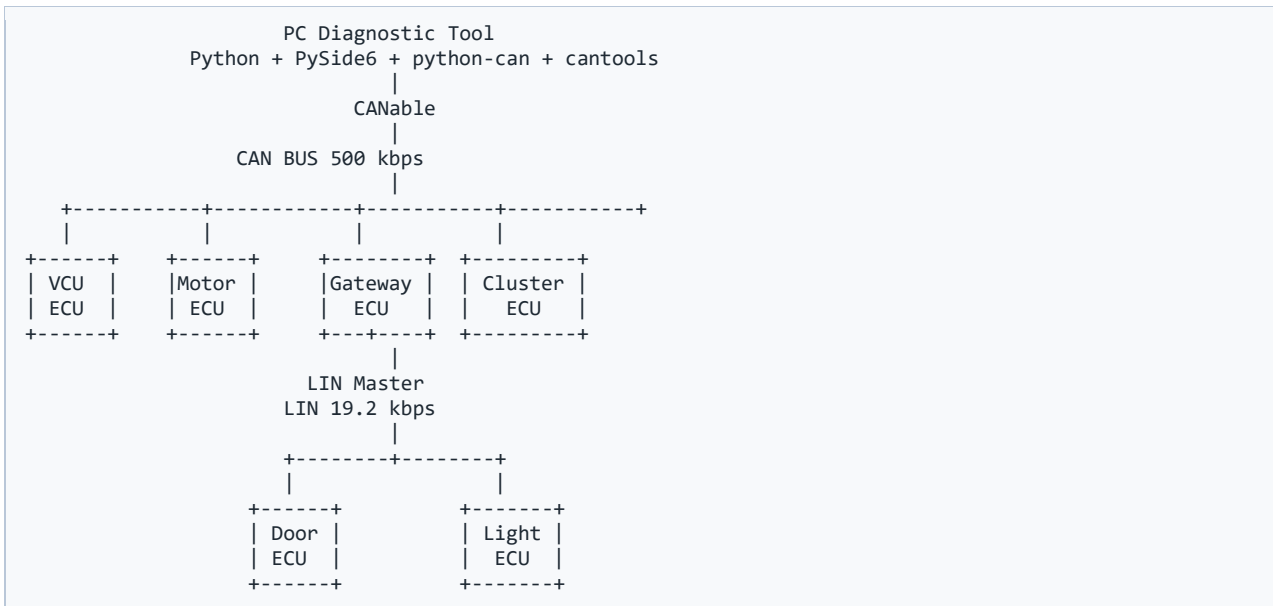
1.2 범위와 명시적 제외

표 1-1. 프로젝트 범위 기준

구분	프로젝트 기준
포함	CAN/LIN Interface 설계, FreeRTOS Gateway, 4 개 CAN ECU, 2 개 LIN ECU, PC Diagnostic Tool, Fault Handling, pytest 기반 자동화
기본안	Motor ECU 는 실제 모터를 필수로 연결하지 않고 Virtual Motor 모델로 동작한다. Torque Request 가 Target RPM 과 RPM 응답으로 변환된다.
선택 확장	실제 모터·인버터 연결, LDF 자동생성, CAN FD, 정식 차량 진단 프로토콜, HIL 장비 연동은 Stretch Goal 로 분리한다.
제외	양산 수준 기능안전 인증, 실제 차량 장착, 고전압 전력부 설계, 상용 진단 규격 전체 구현은 본 MVP 범위에 포함하지 않는다.

1.3 최종 네트워크 아키텍처

그림 1-1. 최종 시스템 구성



PC Diagnostic Tool 은 CANable 을 통해 CAN Bus 에 연결한다. Gateway ECU 는 CAN Node 이면서 동시에 LIN Master 다. Door ECU 와 Light ECU 는 LIN Slave 로 동작하며, LIN Scheduler 가 Frame 전송 시점을 결정한다.

1.4 네트워크 기본 사양

표 1-2. CAN/LIN 사양

항목	CAN	LIN
Bit rate	500 kbps	19.2 kbps
역할	제어·상태·진단 Message 전달	Body ECU Polling 및 상태 수집
Node	VCU, Motor ECU, Gateway ECU, Cluster ECU	Gateway Master, Door ECU Slave, Light ECU Slave
주기 기준	10 ms, 20 ms, 100 ms, event	20 ms Base Schedule, 50 ms/100 ms Frame
PC 연결	CANable -> CAN Bus	직접 연결하지 않고 Gateway 를 통해 관찰
DB 파일	vehicle.dbc	LIN Matrix/Schedule; LDF 는 Stretch Goal

1.5 시스템 동작 흐름

표 1-3. 주요 데이터 경로

흐름	데이터 경로	검증 포인트
----	--------	--------

흐름	데이터 경로	검증 포인트
제어 명령	VCU -> CAN -> Gateway -> Motor/Cluster	TorqueRequest, Gear, Brake, Alive
Body 제어	VCU Gear R -> CAN -> Gateway -> LIN -> Light	ReverseLamp ON
Body 상태	Door -> LIN -> Gateway -> CAN -> Cluster/PC	DoorOpen 표시
장애 전파	Timeout/Alive -> Gateway Health -> CAN Fault_Status	Cluster/PC offline 표시

설계 원칙

인터페이스를 먼저 고정하고 구현을 병렬화한다. CAN Matrix 와 LIN Matrix/Schedule 가 확정되지 않은 상태에서 ECU 별 코딩을 시작하면 같은 Signal 을 서로 다른 의미로 구현하게 되므로, M1 Interface Freeze 를 통과한 뒤 병렬 개발을 시작한다.

2. ECU 역할과 7 인 역할 분담

팀 규모는 7 명이며, 각 담당은 한 개 ECU 만 만드는 데 그치지 않고 자신이 소유한 Interface, 테스트 증적, Jira/Git 이력을 함께 관리한다. System/Gateway 담당은 전체 구조와 통합 품질을 책임지는 조정 역할이다.

2.1 ECU 별 책임

표 2-1. ECU 및 PC Tool 역할

ECU / 구성요소	주요 역할	필수 Interface 와 상태
VCU ECU	Ignition, Accelerator, Brake, Gear 입력을 읽고 Vehicle State Machine 을 관리한다. Torque Request 를 계산해 CAN 으로 송신한다.	CAN Command, 10 ms expected, Timeout/Alive 관리
Motor ECU	Torque Request 를 받아 Target RPM 과 RPM 응답을 계산한다. 기본안은 Virtual Motor 이며 실제 모터는 필수가 아니다.	Motor_Status CAN, RPM, TorqueApplied, MotorState, fail-safe
Gateway ECU	FreeRTOS 기반 CAN-LIN Routing, Signal Cache, LIN Scheduler, Timeout, Alive, Fault, Network Health 를 수행한다.	CAN Node + LIN Master, Routing Table, Health/Fault CAN
Cluster ECU	Speed, RPM, Gear, Door, Light, Fault 상태를 수신해 표시한다. 표시 로직은 상태 수신과 분리한다.	Vehicle_Status, Body_Status, Fault_Status 수신
Door ECU	LIN Slave 로 Open/Close, Lock/Unlock 상태를 제공한다. 응답이 끊기면 Gateway 가 offline 으로 판정한다.	Door_Status 0x10, Door_Command 0x11
Light ECU	LIN Slave 로 Head, Turn, Hazard, Reverse, Brake Lamp 명령을 수행하고 상태를 반환한다.	Light_Command 0x12, Light_Status 0x13
PC Diagnostic Tool	CAN Monitor, DBC Decode, ECU Health, Logging, Fault Injection, Test Automation, 그래프를 제공한다.	CANable, python-can, cantools, PySide6, pytest

2.2 7 인 역할 분담

표 2-2. 7 인 역할 분담

담당	주 소유 영역	핵심 산출물	협업 경계
----	---------	--------	-------

담당	주 소유 영역	핵심 산출물	협업 경계
1. System + Gateway	전체 Architecture, Requirements, FreeRTOS, CAN-LIN Routing, Health/Fault, Integration	system-architecture.drawio/png, gateway-sw-architecture.drawio, Gateway Firmware, Integration Report	모든 ECU Interface 리뷰. 전체 시스템을 조율하고 M0-M5 Gate 를 관리
2. VCU	입력 모델, Vehicle State Machine, Torque Request, CAN Command	VCU Firmware, VCU Test, VCU CAN Evidence	Motor, Gateway 와 Torque/Gear/Alive 의미를 합의
3. Motor	Virtual Motor Model, Target RPM, RPM Response, Motor Status, fail-safe	Motor Firmware, Model Test, Motor_Status Evidence	VCU 의 TorqueRequest 와 Gateway timeout 처리 확인
4. Cluster	Vehicle/Body/Fault 표시, ECU Health 표시	Cluster Firmware, Display Test, Display Evidence	Gateway 가 보내는 Status/Fault Signal 소비
5. Door	Door LIN Slave, Open/Close, Lock/Unlock, response behavior	Door Firmware, LIN Capture, Timeout Test Input	LIN Matrix/Schedule 와 Gateway Master 에 맞춤
6. Light + LIN Common	Light LIN Slave, Head/Turn/Hazard/Reverse/Brake, LIN driver/common	Light Firmware, LIN Common, Schedule Capture	Door 담당과 LIN Frame/Timing 을 공동 리뷰
7. PC Diagnostic + Test	CANable Tool, Python GUI, DBC Decode, Logging, Fault Injection, pytest	diagnostic_gui, tests/, test-result.md, Release Evidence	모든 ECU 의 테스트 입력과 KPI 측정 기준을 조율

System/Gateway 담당의 적합성

전체 요구사항, Interface, RTOS Task, Routing, Fault, Integration 을 동시에 보므로 시스템 전체를 이해하고 여러 담당의 경계를 조정하는 사람에게 적합하다. 단, 모든 코드를 혼자 소유한다는 뜻이 아니라 공통 규칙과 통합 품질을 소유한다는 뜻이다.

2.3 협업 경계

표 2-3. 협업 경계 합의

경계	반드시 합의할 내용	리뷰 주체
VCU <=> Motor	TorqueRequest 단위, Gear/Brake 의미, Motor timeout 시 TorqueApplied=0	VCU 담당 + Motor 담당 + Gateway 담당
VCU <=> Gateway	CAN ID, 주기, Alive Counter, Timeout 30 ms, Signal scaling	VCU 담당 + Gateway 담당
Gateway <=> LIN	PID, Frame 방향, Schedule slot, 3-cycle no response 기준	Gateway 담당 + LIN Common + Door/Light

경계	반드시 합의할 내용	리뷰 주체
Gateway <-> Cluster/PC	Status/Fault Message, online/offline bit, invalid value, logging schema	Gateway 담당 + Cluster + PC/Test

3. 전체 Tool Stack 과 핵심 Tool Chain

도구는 많을수록 좋은 것이 아니라 산출물의 역할이 겹치지 않아야 한다. Jira 는 업무 상태, Git/GitHub 는 실제 변경 이력, Excel 은 설계 초안과 리뷰, CANDb++의 DBC 는 실제 CAN Database 의 기준 파일로 사용한다.

3.1 전체 Tool Stack

표 3-1. 프로젝트 전체 Tool Stack

Tool	용도	사용 단계	주요 산출물 또는 검증
Jira	Task, Sprint, Bug, Test 상태, 담당자, 완료조건	전 단계 / 매주 Sprint	Issue Key, Status, Acceptance Criteria, Bug 이력
Git + GitHub	코드, Branch, Commit, PR, Review, Release, Docs	전 단계	Commit/PR/Review/Tag/Release
draw.io (diagrams.net)	System Architecture, Network Topology, Gateway SW 구조	STEP 2, 13	drawio 원본 + PNG 리뷰본
Markdown + GitHub	Requirements, Convention, Test Plan/Result, README	전 단계	docs/*.md, PR 리뷰 대상 문서
Excel	CAN Matrix, LIN Matrix, Signal 초안, 리뷰용 표	STEP 3, 6, 7	CAN_Matrix.xlsx, LIN_Matrix.xlsx, LIN_Schedule.xlsx
CANDb++	Node, Message, Signal, Value Table 을 정의하고 DBC 생성	STEP 4	vehicle.dbc
cantools	DBC Parse, Encode/Decode, Signal 범위 검증	STEP 5, 22, 23	Validation Log, Python Decoder
canmatrix	DBC 와 Excel/다른 DB 포맷 변환·비교	STEP 4-5 / 변경 검토	변환본, Diff 보조
STM32CubeMX	Clock, GPIO, CAN, UART, LIN UART, FreeRTOS 설정	STEP 8	.ioc, 초기화 코드
STM32CubeIDE	C Firmware 개발, Build, Debug, Flash 설정	STEP 9 이후	.elf/.bin, Firmware Test
ST-LINK	MCU Flash, Debug, Reset, SWD 연결	STEP 9 이후	Flash Log, Debug Evidence
CANable	PC Diagnostic Tool 과 CAN Bus 연결	STEP 11, 22-24	CAN Frame 송수신 경로
SavvyCAN	Raw CAN Monitor, 송수신 주기·ID 관찰	STEP 11, 21, 24	CAN Capture, Cycle/Jitter Evidence

Tool	용도	사용 단계	주요 산출물 또는 검증
USB Logic Analyzer + PulseView	LIN/UART Timing, Break/Sync/PID/Data/Checksum 확인	STEP 12, 24	LIN Capture, Timing Evidence
CoolTerm / Tera Term	UART Log 관찰, 초기 Bring-up 확인	STEP 9-10	Boot/Health/State Log
VS Code	Python Tool, Tests, Markdown, Git 작업	STEP 1 이후	tools/, tests/, docs/ 편집
Python	Diagnostic Tool, Fault Injection, Test Script, KPI 계산	STEP 21-24	스크립트, CSV/JSON Log, 그래프
python-can	CAN Interface 추상화, 송신·수신·Bus 연결	STEP 22-23	CAN API, Test Fixture
PySide6	PC Diagnostic GUI 화면 구성	STEP 22	Vehicle/Health/Network/Raw 화면
pytest	자동화 Test, Regression, Integration Test	STEP 23	PASS/FAIL Report
GitHub Releases	Tag 와 함께 Firmware/DBC/PC Tool/Report 배포	Final Release	v1.0 Release Asset

3.2 핵심 Tool Chain

그림 3-1. 표준 Tool Chain

CAN: Excel CAN Matrix -> CANdb++ -> vehicle.dbc -> cantools -> CANable / SavvyCAN -> ECU Firmware / PC Diagnostic Tool LIN: Excel LIN Matrix -> LIN Schedule -> CubeIDE Firmware -> USB Logic Analyzer / PulseView Firmware: STM32CubeMX -> STM32CubeIDE -> ST-LINK -> CAN / UART / LIN verification Test: Python -> python-can -> cantools -> pytest -> Log / KPI / Regression evidence

표 3-2. Tool Chain 별 기준

구분	기준 흐름	기준 파일
CAN Interface	Excel 에서 리뷰 가능한 Matrix 를 만들고 CANdb++에서 vehicle.dbc 로 확정한다.	network/can/CAN_Matrix.xlsx, vehicle.dbc
LIN Interface	Frame 과 Signal 을 Excel 로 정의하고 Master Schedule 을 별도 파일로 고정한다.	network/lin/LIN_Matrix.xlsx, LIN_Schedule.xlsx
코드	CubeMX 설정을 CubeIDE 프로젝트로 반영하고 ST-LINK 로 Flash 한다.	.ioc, source, build artifact

구분	기준 흐름	기준 파일
검증	Python 이 CANable 을 제어하고 cantools 로 decode 하며 pytest 가 기대값을 판정한다.	tests/, logs/, test-result.md

혼동 금지

Excel 은 설계·리뷰용 원본이고 CANdb++가 생성한 vehicle.dbc 가 실제 CAN Database 다. GitHub 는 파일을 저장하고
변경을 리뷰하는 곳이지 CAN Matrix 의 의미를 자동으로 결정하는 도구가 아니다.

4. Repository 와 산출물 관리

Repository 구조는 담당자별 폴더보다 시스템 산출물의 종류가 먼저 보이도록 구성한다. Firmware 는 ECU 별로 나누되, Interface 와 Test 는 공통 영역에서 관리한다.

4.1 권장 Repository 구조

그림 4-1. 권장 Repository 구조

```
freertos-can-lin-gateway/
|- firmware/
|  |- gateway/
|  |- vcu/
|  |- motor/
|  |- cluster/
|  |- door/
|- light/
|- network/
|  |- can/
|  |  |- CAN_Matrix.xlsx
|  |  |- vehicle.dbc
|  |- lin/
|  |  |- LIN_Matrix.xlsx
|  |  |- LIN_Schedule.xlsx
|- tools/
|- diagnostic_gui/
|- tests/
|  |- can/
|  |- lin/
|  |- gateway/
|- integration/
|- docs/
|  |- requirements.md
|  |- system-architecture.drawio
|  |- system-architecture.png
|  |- gateway-sw-architecture.drawio
|  |- coding-convention.md
|  |- test-plan.md
|  |- test-result.md
|- README.md
|- CONTRIBUTING.md
```

4.2 폴더별 소유권

표 4-1. 산출물 소유권

경로	내용	Primary Owner	변경 시 검토
firmware/gateway	FreeRTOS Task, Routing, Health, LIN Master	System/Gateway	VCU, Door, Light, PC/Test
firmware/vcu	Vehicle State, Torque Request, CAN Command	VCU	Motor, Gateway
firmware/motor	Virtual Motor, Motor Status, fail-safe	Motor	VCU, Gateway, Cluster
firmware/cluster	상태 표시와 Fault 표시	Cluster	Gateway, PC/Test

경로	내용	Primary Owner	변경 시 검토
firmware/door, light	LIN Slave 기능	Door / Light	Gateway + LIN Common
network/can, network/lin	Interface 기준 파일	System/Gateway + 해당 담당	인터페이스 상대 담당 전원
tools, tests	진단, Fault Injection, 자동화	PC Diagnostic/Test	모든 ECU 담당
docs	요구사항, 구조, 규칙, Test Plan/Result	System/Gateway	전체 팀

4.3 변경 추적 흐름

그림 4-2. Interface 변경 통제 흐름

Interface change control:
 Jira Issue -> Excel Matrix update -> Review
 -> CANdb++ / DBC update -> Firmware / Tool update
 -> PR review -> HW / automated test -> Done

Every interface change records:
 reason, old value, new value, affected ECU, affected tests,
 migration note, reviewer, and evidence path.

Repository 운영 기준

README.md 는 팀원이 처음 clone 한 날 바로 실행할 수 있어야 한다. CONTRIBUTING.md 에는 Branch, Commit, PR, Review, Build, Flash, Test 절차를 적는다. 문서가 코드보다 뒤늦게 따라오는 상태를 허용하지 않는다.

5. CAN 설계, CANdb++, DBC 검증

CAN Interface 는 Excel 에서 논의할 수 있어야 하고, CANdb++에서 Node/Message/Signal 의 의미를 확정된 뒤 vehicle.dbc 로 배포한다. DBC 가 만들어졌다고 끝이 아니며, cantools 로 Parse 와 Encode/Decode 를 반복 검증한다.

5.1 CAN Matrix 기본 규칙

- Message ID 는 프로젝트 전체에서 중복되지 않아야 하며, 확장 ID 사용 여부를 명시한다.
- 주기 Message 는 10 ms, 20 ms, 100 ms 등 주기를 고정하고 event Message 는 발생 조건과 재전송 정책을 적는다.
- 모든 Signal 은 Start Bit, Length, Byte Order, Factor, Offset, Min, Max, Unit, Initial, Invalid 를 갖는다.
- Sender 와 Receiver 를 모두 적고, 수신 ECU 가 timeout 과 invalid 를 어떻게 처리할지 별도로 정의한다.
- Alive Counter 는 4 bit, 0-15 순환으로 통일하고 전송 Message 에 포함할지 규칙을 고정한다.

5.2 CAN Message 예시

표 5-1. CAN Matrix 예시

Message	CAN ID	주기	DLC	Sender / Receiver	Purpose
VCU_Command	0x100	10 ms	8	VCU -> Motor, Gateway, Cluster	TorqueRequest, Gear, Brake, AliveCounter
Motor_Status	0x110	10 ms	8	Motor -> VCU, Gateway, Cluster	RPM, TorqueApplied, MotorState, AliveCounter
Vehicle_Status	0x120	20 ms	8	Gateway -> Cluster, PC	Speed, RPM, Gear, Brake, network state
Body_Status	0x200	100 ms	8	Gateway -> Cluster, PC	Door/Light 상태, ReverseLamp, DoorOpen
Gateway_Status	0x300	100 ms	8	Gateway -> all/PC	Gateway Health, Queue usage, Alive
Fault_Status	0x310	event	8	Gateway -> Cluster, PC	Fault code, ECU offline, recovery state

5.3 Signal 정의 예시

표 5-2. CAN Signal 예시

Message.Signal	Start / Length	Byte order	Factor / Offset	Min / Max	Unit	Initial / Invalid
VCU_Command.TorqueRequest	0 / 16	Intel	0.1 / 0	0 / 4000	Nm	0 / 0xFFFF
VCU_Command.Gear	16 / 3	Intel	1 / 0	0 / 5	enum	P / 7
VCU_Command.Brake	19 / 1	Intel	1 / 0	0 / 1	bool	0 / 1
VCU_Command.AliveCounter	20 / 4	Intel	1 / 0	0 / 15	count	0 / 0xF
Motor_Status.RPM	0 / 16	Intel	1 / 0	0 / 12000	rpm	0 / 0xFFFF
Body_Status.ReverseLamp	0 / 1	Intel	1 / 0	0 / 1	bool	0 / 1
Gateway_Status.GatewayAlive	0 / 4	Intel	1 / 0	0 / 15	count	0 / 0xF

왜 필드가 모두 필요한가

Start Bit 와 Length 는 비트 배치를 재현하고, Byte Order 는 Intel/Motorola 해석을 결정한다. Factor 와 Offset 은 raw 값과 physical 값의 변환을 고정한다. Min/Max, Unit, Initial, Invalid 가 없으면 GUI 와 Test 가 같은 Message 를 서로 다른 의미로 처리하게 된다.

5.4 CANdb++ 사용법과 역할

CANdb++ 기본 무료 버전을 사용한다는 전제로 진행한다. 이 프로젝트에는 CANdb++ Admin 이 필요하지 않다. Admin 은 별도의 관리 기능이 필요한 조직형 환경을 위한 선택 사항이며, 팀 프로젝트의 Node/Message/Signal 정의와 DBC 생성은 기본 CANdb++로 수행한다.

표 5-3. CANdb++ 객체 매핑

CANdb++ 객체	정의 내용	프로젝트 예
Node	통신에 참여하는 ECU 이름과 송수신 주체	VCU, MOTOR, GATEWAY, CLUSTER
Message	CAN ID, DLC, 송신 Node, 주기 또는 event	VCU_Command 0x100 10 ms
Signal	Start Bit, Length, Byte Order, Factor, Offset, Min/Max, Unit	TorqueRequest 16 bit factor 0.1
Value Table	숫자 값과 사람이 읽는 상태의 매핑	Gear: 0=P, 1=N, 2=D, 3=R

CANdb++ 객체	정의 내용	프로젝트 예
Sender / Receiver	Signal 또는 Message 를 누가 보내고 누가 소비하는지	VCU -> Gateway/Motor/Cluster

그림 5-1. CANdb++ 작성 순서

CANdb++ authoring sequence:

1. Create database and add nodes:
VCU, MOTOR, GATEWAY, CLUSTER
2. Create message:
VCU_Command / ID 0x100 / DLC 8 / cycle 10 ms
3. Add signals:
Start Bit, Length, Byte Order, Factor, Offset,
Min, Max, Unit, Initial, Invalid
4. Add value tables and Sender / Receiver
5. Repeat for all messages in CAN_Matrix.xlsx
6. Save as network/can/vehicle.dbc

5.5 Excel, CANdb++, DBC, GitHub 의 관계

표 5-4. CAN Interface 기준 흐름

단계	행동	완료 결과
설계	Excel CAN Matrix 에서 ID, 주기, Signal, 범위, Receiver 를 작성	팀이 읽고 리뷰할 수 있는 초안
Review	VCU/Motor/Gateway/Cluster 와 PC/Test 담당이 의미와 방향을 검토	합의된 Matrix
생성	CANdb++에서 Node, Message, Signal, Value Table 을 입력	vehicle.dbc
검증	cantools 와 canmatrix 로 Parse/Encode/Decode/변환 비교	Validation Log
관리	Excel 과 DBC 를 같은 Jira Issue/Branch/PR 로 커밋	변경 이력과 리뷰 증적

변경 통제

CAN 변경은 Excel 수정 -> Review -> CANdb++ 수정 -> DBC 수정 -> Git PR 순서로 진행한다. DBC 파일만 바꾸고 Matrix 를 바꾸지 않거나, Matrix 만 바꾸고 코드가 따라오지 않는 상태를 허용하지 않는다. 모든 DBC 변경은 Jira Issue, Branch, Commit, PR 로 관리한다.

5.6 cantools 검증 예시

코드 5-1. cantools CLI 및 Python Encode/Decode

```
# CLI: inspect the generated database
python -m cantools dump vehicle.dbc

# Python: encode and decode a VCU command
import cantools

db = cantools.database.load_file("vehicle.dbc")
msg = db.get_message_by_name("VCU_Command")
payload = msg.encode({
    "TorqueRequest": 120.0,
    "Gear": 3,
    "Brake": 0,
    "AliveCounter": 7,
})
decoded = msg.decode(payload)
assert decoded["Gear"] == 3
assert decoded["AliveCounter"] == 7
print(payload.hex(), decoded)
```

표 5-5. DBC Validation 기준

검증 항목	통과 기준	실패 시 조치
Parse error	vehicle.dbc 를 오류 없이 load_file 할 수 있다.	CANdb++ 문법과 객체 정의 확인
ID / DLC	Matrix 와 DBC 의 ID, DLC, Sender 가 일치한다.	Excel 과 CANdb++를 함께 수정
Signal overlap	같은 Message 안에서 Signal 이 겹치지 않는다.	Start Bit/Length 재검토
Factor / Offset	raw <-> physical 변환 결과가 요구 범위와 일치한다.	Factor, Offset, Min/Max 검토
Encode / Decode	대표 payload 가 기대 Signal 로 왕복 변환된다.	Byte Order, signedness, scale 검토
Value Table	Gear 등 enum 값이 코드·GUI·DBC 에서 동일하다.	Value Table 과 test fixture 수정

5.7 canmatrix 의 사용 위치

canmatrix 는 DBC 를 다른 포맷으로 변환하거나 두 Database 를 비교하는 보조 도구로 사용한다. 기존 파일을 canmatrix 가 대신 결정하지는 않는다. 변경 PR 에는 변환 결과 또는 Diff 를 첨부해 Matrix 와 DBC 사이의 차이를 빠르게 발견한다.

6. LIN Matrix 와 Schedule 설계

LIN에서는 Gateway가 Master이고 Door ECU와 Light ECU가 Slave다. LIN Matrix는 Frame과 Signal의 의미를 정의하고, LIN Schedule은 Master가 어느 시점에 어떤 Frame을 Polling하는지 정의한다. LDF는 이번 프로젝트의 필수가 아니며, 필요할 때만 Stretch Goal로 추가한다.

6.1 LIN Matrix 예시

표 6-1. LIN Matrix 예시

Frame	PID	주기	Publisher	Subscriber	Signal / 목적
Door_Status	0x10	100 ms	Door ECU	Gateway	DoorOpen, DoorLocked, DoorAlive
Door_Command	0x11	100 ms	Gateway	Door ECU	LockCommand, RequestState
Light_Command	0x12	50 ms	Gateway	Light ECU	HeadLamp, TurnLamp, HazardLamp, ReverseLamp, BrakeLamp
Light_Status	0x13	100 ms	Light ECU	Gateway	LampState, LightAlive, DiagnosticState

6.2 LIN Schedule 예시

표 6-2. LIN Schedule 예시

Slot time	Schedule entry	Frame	의미
0 ms	1	Door_Status	Door 상태를 먼저 수집
20 ms	2	Light_Command	Lamp command를 주기적으로 전달
40 ms	3	Light_Status	Lamp 응답을 수집
60 ms	4	Door_Command	Lock 또는 Request 상태 전달
100 ms	repeat	Schedule 0으로 반복	20 ms Base Schedule의 반복 기준

그림 6-1. LIN Master Schedule

LIN Master schedule, 20 ms base tick:

```
t=0   Door_Status
t=20  Light_Command
t=40  Light_Status
t=60  Door_Command
t=100 repeat from t=0
```

Door ECU: responds to Door_Status / accepts Door_Command

Light ECU: responds to Light_Status / accepts Light_Command

6.3 LIN 구현 규칙

표 6-3. LIN 구현 기준

규칙	내용	검증
Master only	Schedule 을 실행하고 Header 를 시작하는 주체는 Gateway 다.	Gateway log + PulseView
Slave response	해당 PID 를 받은 ECU 만 Response 를 제공한다.	Door/Light capture
No response	3 cycle 연속 응답이 없으면 해당 ECU offline 으로 판정한다.	LIN timeout test
Timing	19.2 kbps 와 Slot 간격을 실제 Capture 로 확인한다.	PulseView time measurement
LDF	이번 프로젝트 필수 아님. Frame/Signal/Schedule 표만으로 시작한다.	Stretch Goal 로 별도 Jira

LIN bring-up 순서

먼저 Gateway Master 가 고정 Schedule 을 실행하는지 확인하고, 그다음 Light ECU 단일 Response, Door ECU 단일 Response, 두 ECU 동시 Schedule, 마지막으로 no response timeout 을 순서대로 검증한다.

7. MCU Bring-up 과 UART Logging

MCU 설정은 STM32CubeMX 에서 시작하고, STM32CubeIDE 에서 Build 와 Debug 를 수행하며, ST-LINK 로 Flash 한다. CAN/LIN 을 바로 붙이기 전에 Clock, GPIO, UART, LED, Scheduler Tick 부터 확인하면 원인 분리가 쉬워진다.

7.1 CubeMX Peripheral 설계

표 7-1. MCU Peripheral 계획

Peripheral	설정 방향	Bring-up 확인
Clock	CAN/LIN/UART baud 와 FreeRTOS tick 이 계산 가능한 clock tree	Clock log 또는 UART banner
GPIO	CAN transceiver enable, LIN enable, LED, button/input mock	pin toggle / LED
CAN	500 kbps, Filter, RX interrupt/FIFO, TX mailbox	SavvyCAN raw frame
UART	debug log 용 baud 와 DMA/interrupt 선택	CoolTerm/Tera Term 에서 banner
LIN UART	19.2 kbps, Break/Sync/PID/Response 를 다룰 수 있는 driver	PulseView capture
FreeRTOS	Task, Queue, Semaphore/Mutex, Heap 정책	task heartbeat, queue high-water
Timer	10 ms/20 ms/100 ms 주기와 LIN base tick	timestamp drift / jitter

7.2 Firmware Bring-up 순서

그림 7-1. Firmware Bring-up Gate

```
Bring-up gate:
1. CubeMX .ioc opens and generates cleanly
2. CubeIDE build PASS with warnings reviewed
3. ST-LINK flash and reset PASS
4. UART boot banner appears
5. LED / GPIO heartbeat toggles
6. FreeRTOS scheduler starts
7. CAN loopback or single-frame test PASS
8. LIN UART byte/timing capture PASS
9. Application tasks start one by one
```

표 7-2. Bring-up 증적

증적	최소 내용	보관 위치
Build	Project, board, configuration, warning count, binary hash	Jira attachment / GitHub Actions or log
Flash	Board, ST-LINK, time, success result	docs/test-result.md
UART	Boot banner, firmware version, task heartbeat	UART capture or text log
Bus	CAN raw frame or LIN waveform with measured timing	tests/evidence or release asset

7.3 UART Logging 표준

UART Logging 은 초기 Bring-up 과 Fault 원인 분석에 사용한다. ISR 에서는 문자열 포매팅과 blocking printf 를 수행하지 않고, 이벤트 또는 짧은 구조체만 LoggerTask 로 넘긴다.

코드 7-1. UART Logging 형식 예시

```
[00001234][GW][INFO][CANRxTask] id=0x100 dlc=8 alive=7
[00001235][GW][INFO][RoutingTask] VCU_Command.Gear=R
[00001236][GW][INFO][LINScheduler] frame=Light_Command slot=20
[00001550][GW][WARN][HealthTask] VCU_TIMEOUT age_ms=31
[00001551][GW][FAULT][FaultManager] code=VCU_TIMEOUT action=TORQUE_ZERO
```

표 7-3. UART Log schema

필드	규칙	예
timestamp	Monotonic ms. Wall clock 보다 순서와 지연 분석을 우선	00001234
module	ECU 또는 Gateway module 약어	GW, VCU, MOTOR
level	DEBUG/INFO/WARN/FAULT. Release 에서는 DEBUG 를 제한	WARN
event	검색 가능한 대문자 또는 CamelCase event	VCU_TIMEOUT
payload	key=value 형태. 의미가 바뀌면 문서와 Test 도 수정	age_ms=31

로그 운영 주의

로그는 기능의 대체물이 아니다. CAN 주거나 LIN Timing 을 UART 출력 시점으로 추정하지 말고, bus timestamp 또는 Logic Analyzer 의 실제 edge 를 사용해 KPI 를 측정한다.

8. Gateway FreeRTOS 구조와 Routing

Gateway 는 두 네트워크 사이의 의미 변환과 상태 감시를 동시에 수행한다. FreeRTOS 는 CAN 수신, Routing, LIN Schedule, Health, Logger 를 독립적인 시간 기준과 우선순위로 분리해 데이터 경로를 설명 가능하게 만든다.

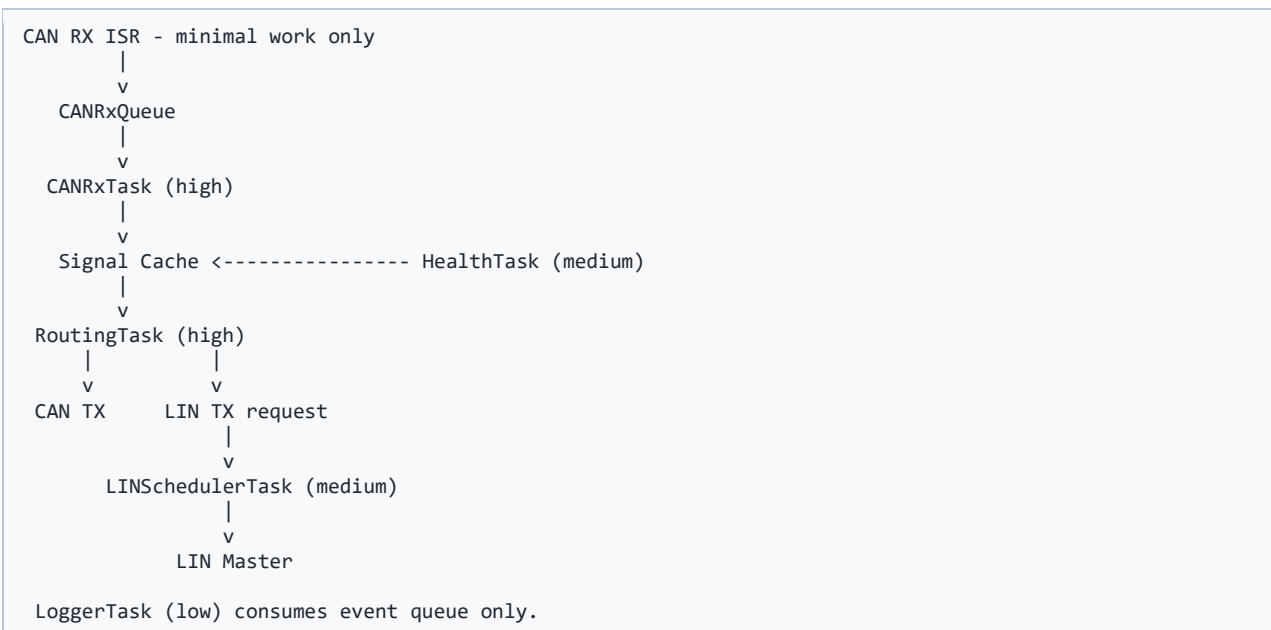
8.1 FreeRTOS 를 사용하는 이유

표 8-1. FreeRTOS 선택 이유

이유	프로젝트에서의 효과
시간 분리	10 ms CAN RX 와 20 ms LIN Schedule, 100 ms Health 를 각각 Task 주기로 표현
우선순위	CAN RX 와 Routing 을 high, LIN Scheduler/Health 를 medium, Logger 를 low 로 관리
Queue 기반 전달	ISR 은 최소 작업만 하고 Queue 로 Task 에 전달해 ISR latency 를 줄임
관찰 가능성	Task heartbeat, Queue high-water, dropped frame 을 KPI 로 수집
확장성	진단·로깅·추가 Routing 이 기존 bus ISR 을 복잡하게 만들지 않음

8.2 Gateway SW 구조

그림 8-1. Gateway FreeRTOS data path



8.3 Task, Queue, Semaphore, Mutex 설계

표 8-2. Gateway RTOS Object

Object	Priority / type	책임	공유 데이터 규칙
CANRxTask	high / Task	CANRxQueue 를 읽고 Message ID/DLC 검증, Signal Cache 업데이트	Cache write 는 Task context 에서 수행
RoutingTask	high / Task	Cache 변화를 Routing Table 로 평가해 CAN/LIN TX 요청 생성	Routing input snapshot 사용
LINSchedulerTask	medium / Task	20 ms base tick 으로 LIN Frame Polling, response timeout 관리	Schedule 은 read-only
HealthTask	medium / Task	age, alive, queue usage, ECU online/offline, Fault state 갱신	Health state 는 Gateway 가 소유
LoggerTask	low / Task	이벤트를 UART/optional ring buffer 로 출력	blocking log 금지
CANRxQueue	Queue	ISR -> CANRxTask raw frame 전달	full 이면 drop counter 증가
RoutingQueue	Queue	RoutingTask -> TX worker 또는 scheduler 요청	복사 또는 ownership 규칙 명시
EventQueue	Queue	Health/Fault/Logger 이벤트 전달	critical fault 는 drop 우선순위 금지
Mutex	필요 시	PC/diagnostic shared state 등 실제 다중 writer 가 있을 때만	무분별한 global mutex 금지
Semaphore	필요 시	DMA/ISR 완료 또는 단일 자원 synchronization	사용 목적을 설계 문서에 기록

Queue/Semaphore/Mutex 규칙

Queue, Semaphore, Mutex 는 이름이 있어야 할 뿐 아니라 목적이 있어야 한다. 각각의 생성 위치, 길이, item size, producer/consumer, overflow 정책을 gateway-sw-architecture.drawio 또는 코드 주석과 Test Plan 에 남긴다.

8.4 Signal Cache 와 Routing Table

Routing 은 if 문이 ECU 별로 난립하는 구조를 피하고, source network/message/signal -> conversion -> destination network/message/signal 의 행(row)으로 관리한다. 변환과 유효성 판단을 Table entry 또는 작은 함수로 분리하면 Interface 변경이 쉬워진다.

표 8-3. Routing Table 예시

Source	Conversion / condition	Destination	설명
CAN VCU_Command.Gear	if Gear == R	LIN Light_Command.ReverseLamp=1	Gear R 이면 후진등 ON
LIN Door_Status.DoorOpen	bool copy + debounce	CAN Body_Status.DoorOpen=1	Door 상태를 Cluster/PC 에 전달
CAN Motor_Status.RPM	scale / range check	CAN Vehicle_Status.RPM	Cluster 표시용 상태로 전달
Gateway Health.VCU_TIMEOUT	fault map	CAN Fault_Status.Code=VCU_TIMEOUT	offline 상태를 외부에 공지

코드 8-1. routing_table.c/h 권장 구조

```
// routing_table.h
typedef struct {
    Network source_network;
    uint16_t source_id;
    SignalId source_signal;
    ConvertFn convert;
    Network destination_network;
    uint16_t destination_id;
    SignalId destination_signal;
} RoutingEntry;

// routing_table.c
static const RoutingEntry routing_table[] = {
    { CAN, 0x100, SIG_GEAR, gear_to_reverse_lamp,
      LIN, 0x12, SIG_REVERSE_LAMP },
    { LIN, 0x10, SIG_DOOR_OPEN, bool_copy,
      CAN, 0x200, SIG_DOOR_OPEN },
};
```

8.5 Gateway MVP 통과조건

표 8-4. Gateway MVP 통과조건

MVP	시나리오	필수 관찰값	통과조건
MVP-1	VCU Gear=R -> CAN -> Gateway decode/routing -> LIN -> Light	Light_Command.ReverseLamp=1, Light ECU lamp state ON	SavvyCAN 과 PulseView 에서 양쪽 경로를 확인하고 Light 상태를 PC/Cluster 에 표시
MVP-2	DoorOpen -> LIN -> Gateway -> CAN -> Cluster	Door_Status.DoorOpen=1, Body_Status.DoorOpen=1	Door 상태가 Cluster 와 PC 에 DOOR OPEN 으로 표시

MVP 판단

위 두 시나리오가 성공하면 Gateway 가 단순 CAN/LIN Driver 가 아니라 의미 기반 Signal Routing 을 수행한다는 것을 증명한다. 이후 Fault, 성능, PC Tool, 자동화가 이 MVP 의 회귀 테스트가 된다.

9. ECU 애플리케이션 구현

각 ECU 는 Interface Freeze 이후 병렬 개발한다. 먼저 입력과 상태를 deterministic 하게 만들고, 그다음 실제 bus 연결, Fault 처리, Cluster/PC 표시를 붙인다.

9.1 VCU Application

표 9-1. VCU 구현 기준

기능	구현 기준	CAN/검증
Ignition	OFF, ON, RUN 상태를 관리하고 startup 시 ECU online 상태를 준비	Vehicle_Status 또는 Gateway Health
Accelerator	0-100% 입력 모델. PC 또는 mock input 으로 대체 가능	TorqueRequest 계산 Test
Brake	bool 또는 pressure mock. Brake=1 이면 TorqueRequest 를 제한 또는 0	VCU_Command.Brake
Gear	P/N/D/R state. Value Table 과 동일한 enum 사용	VCU_Command.Gear
State machine	IGN_OFF -> READY -> DRIVE -> FAULT 전이와 금지 전이를 정의	State transition Test
Alive/timeout	VCU_Command 에 AliveCounter 0-15 순환. Motor/Gateway 응답 age 를 관찰	30 ms timeout 경로

9.2 Virtual Motor ECU

실제 모터는 프로젝트 필수가 아니다. 기본안에서는 Virtual Motor ECU 가 VCU 의 TorqueRequest 를 받아 target RPM 을 계산하고, 1 차 지연 응답 모델로 현재 RPM 을 만든다. 이 모델은 Gateway 와 Cluster 의 데이터 경로를 검증하기에 충분하다.

코드 9-1. Virtual Motor 응답 모델

```
target_rpm = clamp(torque_request * K_TORQUE_TO_RPM, 0, RPM_MAX)
rpm_error = target_rpm - rpm_prev
rpm_next = rpm_prev + ALPHA * rpm_error

if vcu_timeout or invalid_torque or fault_state:
    torque_applied = 0
    target_rpm = 0
    rpm_next = decay_to_zero(rpm_prev)
else:
    torque_applied = torque_request

Motor_Status = { RPM, TorqueApplied, MotorState, AliveCounter }
```

표 9-2. Virtual Motor 데이터 흐름

입력	계산	출력
----	----	----

입력	계산	출력
TorqueRequest	factor/offset decode, range check	target RPM
Brake / Gear	금지 상태면 target=0 또는 limit	MotorState
VCU timeout	fail-safe path	TorqueApplied=0, RPM decay
Model parameter	K_TORQUE_TO_RPM, ALPHA, RPM_MAX	재현 가능한 RPM response

9.3 Cluster ECU

Cluster 는 제어 계산보다 표시와 상태 해석에 집중한다. Vehicle_Status 에서 Speed/RPM/Gear/Brake 를, Body_Status 에서 Door/Light 를, Fault_Status 에서 Fault Code 와 ECU offline 을 수신해 현재 화면을 만든다. 수신 age 가 오래되면 표시를 stale 또는 offline 으로 바꾸는 규칙을 둔다.

9.4 Door ECU

Door ECU 는 LIN Slave 다. Door Open/Close 와 Lock/Unlock 상태를 내부 상태로 갖고, Gateway 의 Door_Command 에 따라 lock command 를 적용한다. Door_Status 에는 DoorOpen, DoorLocked, DoorAlive 를 포함하고, 테스트에서는 응답 중단을 의도적으로 만들 수 있어야 한다.

9.5 Light ECU

Light ECU 는 LIN Slave 로 Head, Turn, Hazard, Reverse, Brake Lamp 를 관리한다. Light_Command 의 각 bit 를 실제 출력 또는 LED/mock output 에 매핑한다. 우선순위는 Hazard, Brake, Reverse 등 프로젝트 요구사항에 따라 문서화하되, MVP 에서는 ReverseLamp ON/OFF 경로를 먼저 완성한다.

표 9-3. Light ECU 기능

Lamp	입력	표시/검증
Head	Light_Command.HeadLamp	LED/mock + Light_Status
Turn	Light_Command.TurnLamp	left/right 또는 enum
Hazard	Light_Command.HazardLamp	양쪽 동시
Reverse	Light_Command.ReverseLamp	Gear=R MVP. 반드시 PulseView 와 Status 로 확인
Brake	Light_Command.BrakeLamp	VCU Brake 와 연동

10. Fault Handling 과 Fault Injection

Fault Handling 은 예외 상황을 출력하는 기능이 아니라, 감지 기준, 상태 전이, fail-safe action, 외부 전파, 복구 조건을 하나의 상태 머신으로 정의하는 일이다.

10.1 Fault detection 기준

표 10-1. Fault 감지·대응 기준

Fault 시나리오	감지 기준	Fault Code / Action	외부 전파
VCU CAN timeout	VCU_Command expected 10 ms. age 가 30 ms 를 넘으면 timeout	VCU_TIMEOUT / Motor torque=0, Gateway health degraded	Fault_Status -> Cluster/PC
Door LIN timeout	Door_Status 100 ms Frame 이 3 cycle 연속 no response	DOOR_ECU_OFFLINE / Door state invalid	Body_Status + Fault_Status -> Cluster/PC
Alive counter	0..15 순환. 기대값과 다음 값이 불연속이면 error	ALIVE_ERROR / 해당 Message invalid	Fault_Status + log
ECU offline	상위 fault 또는 상태 age 가 offline threshold 초과	ECU_OFFLINE / 표시 offline	Gateway_Status, Fault_Status
Queue drop	CANRxQueue 또는 EventQueue full 발생	QUEUE_DROP / KPI fail 또는 degraded	Gateway_Status + log

10.2 Timeout 과 Alive 상세

그림 10-1. Timeout/Alive 규칙

```

VCU_Command:
  expected period = 10 ms
  timeout threshold = 30 ms
  result = VCU_TIMEOUT

Door_Status:
  expected period = 100 ms
  no response for 3 cycles
  result = DOOR_ECU_OFFLINE

AliveCounter:
  values = 0,1,2,...,15,0,1,...
  next = (previous + 1) mod 16
  discontinuity => ALIVE_ERROR
  
```

Fail-safe

VCU 가 offline 이면 Motor ECU 는 torque=0 을 적용한다. Door ECU 가 offline 이면 Door 상태를 invalid/offline 으로 표시하고 Fault 를 CAN 으로 Cluster 와 PC 에 전달한다. Fault 를 감지했지만 외부에 알리지 않는 구현은 통과로 보지 않는다.

10.3 Fault State 와 복구

표 10-2. Fault State Machine

상태	진입	동작	복구
NORMAL	모든 age/alive 정상	주기 송수신, 정상 표시	N/A
DEGRADED	첫 timeout 또는 queue drop	Fault 기록, display stale 가능	정상 Frame/Response 가 정해진 횟수 연속 수신
FAIL_SAFE	VCU timeout 또는 critical fault	Motor torque=0, 위험 명령 차단	VCU online + valid data + state reset
OFFLINE	3-cycle LIN no response 또는 ECU age 초과	offline 표시, 대체값/invalid 사용	ECU response 복구와 alive 정상 확인

10.4 Fault Injection 설계

Fault Injection 은 PC Tool 또는 테스트 스크립트가 특정 Frame 을 생략하거나 Alive 를 깨뜨리거나 잘못된 Signal 을 넣어 Fault 감지와 fail-safe 동작을 재현하는 기능이다. 주기, 길이, 복구 조건을 Test Case 에 고정한다.

코드 10-1. Fault Injection 예시

```
def inject_vcu_timeout(bus, duration_ms=50):
    # Do not send VCU_Command for longer than 30 ms.
    wait_ms(duration_ms)

def inject_alive_error(bus):
    send_vcu_command(bus, gear="D", alive=7)
    send_vcu_command(bus, gear="D", alive=12) # discontinuity

def inject_door_offline(bus):
    # Stop Door ECU response for 3 Door_Status cycles.
    stop_lin_response("Door ECU")
    wait_ms(300)
    assert_can_fault("DOOR_ECU_OFFLINE")
```

10.5 Fault 전파 경로

그림 10-2. Fault 전파 경로

```
VCU timeout:
    missing CAN frame -> Gateway age check -> VCU_TIMEOUT
    -> Motor torque=0 -> Fault_Status -> Cluster + PC

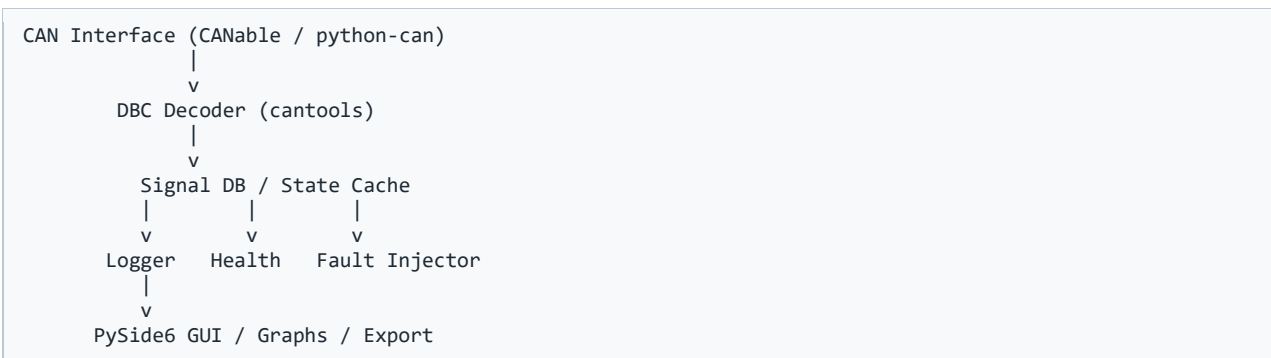
Door offline:
    missing LIN response x3 -> Gateway LIN health
    -> DOOR_ECU_OFFLINE -> Body_Status invalid
    -> Fault_Status -> Cluster + PC
```

11. PC Diagnostic Tool

PC Diagnostic Tool 은 CANable 을 통해 CAN Bus 에 연결하고, vehicle.dbc 를 기준으로 raw CAN frame 을 decode 한다. GUI 는 관찰과 테스트를 쉽게 만드는 수단이며, 진단 로직과 DBC Decode 를 화면 코드에 섞지 않는다.

11.1 GUI 구조

그림 11-1. PC Tool 내부 구조



11.2 화면과 기능

표 11-1. PC Diagnostic Tool 화면

화면	표시 항목	동작
Vehicle Status	speed, rpm, gear, brake, ignition, torque	현재 상태와 마지막 수신 age
ECU Health	VCU, Motor, Gateway, Cluster, Door, Light online/offline	health bit, timeout, last-seen
Network Status	CAN frames/s, CAN timeout, LIN timeout, queue drop	실시간 KPI 와 threshold 표시
Raw CAN Monitor	timestamp, ID, DLC, payload	Filter, pause, save capture
DBC Decode	Message name, Signal, physical value, unit	vehicle.dbc load 및 decode
Fault Injection	drop frame, wrong alive, invalid value, LIN response stop	시나리오 선택 후 결과 확인
Logging / Graph	CSV/JSON 저장, RPM/Torque/Health graph	test-result.md 와 연결할 evidence export

11.3 Signal DB 와 Logging schema

표 11-2. PC Tool Logging schema

필드	설명	예
timestamp_ms	CAN 또는 monotonic timestamp	1234567
bus	CAN / derived LIN	CAN
message	DBC Message name	Vehicle_Status
signal	decoded Signal name	RPM
value	physical value	1250
unit	DBC unit	rpm
quality	valid / stale / invalid / offline	valid

11.4 개발 순서

- CLI raw CAN monitor 를 먼저 구현해 GUI 없이 Bus 와 DBC 를 확인한다.
- cantools decode 와 Signal Cache 를 구현한 뒤 Vehicle Status 와 ECU Health 를 붙인다.
- Logging 과 export 를 구현하고, Fault Injection 을 별도 모듈로 분리한다.
- PySide6 화면은 CAN Interface, Decoder, Logger 를 호출하는 thin UI 로 유지한다.
- pytest 에서 GUI 가 아닌 Decoder/State/Fault logic 을 자동화하고, GUI 는 smoke test 와 manual evidence 로 보완한다.

12. Automated Test 와 KPI 측정

테스트는 Unit/Interface/Integration/HW 의 층으로 나누고, 각 Test 가 어떤 Jira Issue 의 완료조건인지 기록한다. 숫자가 필요한 KPI 는 목표와 실제 측정 결과를 구분해 기록한다.

12.1 Test 구조

표 12-1. Test 층

층	대상	도구	예시
Unit	Signal conversion, Virtual Motor, state machine	pytest / pure Python or host C	test_motor.py, test_vcu.py
Interface	DBC encode/decode, CAN/LIN frame fields	cantools, pytest	test_routing.py
Integration	VCU -> Gateway -> Motor/Cluster, Door -> Gateway	python-can + CANable + real boards	test_gateway.py, test_gateway_mvp.py
Fault	timeout, alive, offline, fail-safe	Fault Injection + pytest	test_can_timeout.py, test_lin_timeout.py
Performance	cycle, jitter, latency, queue use	SavvyCAN, PulseView, Python	KPI CSV/graph

12.2 pytest Integration 예시

코드 12-1. pytest 에서 Gear R -> ReverseLamp 확인

```
def test_reverse_lamp_routing(bus, decoded):
    send_vcu_command(bus, gear="R", torque_request=0, alive=1)

    body_status = wait_for_message(
        bus, name="Body_Status", timeout=0.5
    )
    decoded_body = decoded.decode_message("Body_Status", body_status.data)

    assert decoded_body["ReverseLamp"] == 1
```

12.3 권장 Test 파일

표 12-2. Test 파일 예시

파일	검증 내용	완료 증적
test_vcu.py	Ignition, Gear, Brake, State Machine, TorqueRequest	pytest output + VCU log
test_motor.py	TorqueRequest -> target RPM -> RPM response, fail-safe	expected/actual RPM table
test_gateway.py	CAN RX, Cache, CAN TX, LIN Master, Task health	Gateway log + capture

파일	검증 내용	완료 증적
test_routing.py	source -> conversion -> destination	routing case matrix
test_can_timeout.py	VCU_Command 30 ms timeout	VCU_TIMEOUT + torque 0
test_lin_timeout.py	Door 3-cycle no response	DOOR_ECU_OFFLINE
test_alive_counter.py	0-15 wrap and discontinuity	ALIVE_ERROR
test_gateway_mvp.py	Reverse Lamp, Door Open two MVP paths	MVP evidence

12.4 KPI 및 측정 도구

표 12-3. KPI 측정 기준

KPI	예시 목표 / 판정	측정 도구	실제 결과 기록
CAN cycle / jitter	10 ms Message. jitter <= +/-1 ms 권장	SavvyCAN + Python timestamp	평균/최소/최대/표준편차를 숫자로 기록
CAN packet loss	정상 링크에서 0% 목표	SavvyCAN capture count	송신 기대 개수와 수신 개수 비교
LIN schedule / timing	20 ms base, slot jitter <= +/-1 ms 권장	PulseView	각 Frame slot 실제 시각
CAN -> LIN latency	Gear R 송신부터 Light Command 까지 <= 20 ms 예시 목표	CANable + Logic Analyzer timestamp	P50/P95/P99 또는 min/max
CAN timeout	VCU expected 10 ms, 30 ms 에 VCU_TIMEOUT	Python injection + UART/CAN	감지 시각과 threshold
LIN timeout	Door 100 ms, 3 cycle no response	PulseView + Fault CAN	offline 판정 시간
Alive fault	불연속 1 회에 ALIVE_ERROR	Python wrong counter	감지 여부와 복구 여부
Queue usage/drop	high-water < 70% 권장, nominal drop=0	Gateway_Status + UART	peak usage, drop count
ECU recovery	통신 복구 후 정상 상태 전환 시간	Fault injection + Python	fault clear 까지 ms

KPI 기록 원칙

목표 숫자를 결과처럼 쓰지 않는다. Release Report 에는 테스트 날짜, Bus 설정, 보드, 측정 도구, 샘플 수, 최소/최대/평균 또는 P95, 실패 수를 실제 숫자로 제시한다.

13. 12 주 Roadmap 과 Milestones

12 주 일정은 한 주 단위의 Gate 를 두고, 주말 전체 통합을 권장한다. 각 주의 산출물과 Gate 가 다음 주의 입력이 되도록 만든다.

13.1 12 주 Roadmap

표 13-1. 12 주 Roadmap

주차	목표	사용 Tool	산출물	Gate
1	Requirements / Architecture	Jira, draw.io, Markdown	requirements.md, topology, 역할표	M0 후보. 범위와 ECU 역할 리뷰
2	CAN/LIN Interface 설계	Excel, Jira, GitHub	CAN Matrix, LIN Matrix, Schedule 초안	Signal 과 주기 리뷰 완료
3	MCU Bring-up	CubeMX, CubeIDE, ST-LINK, UART	.ioc, boot log, GPIO/CAN/LIN skeleton	각 보드 flash 와 UART banner
4	CAN Network	CANdb++, cantools, CANable, SavvyCAN	vehicle.dbc, CAN capture, validation log	M1 Interface Freeze + CAN raw PASS
5	LIN Network	CubeIDE, PulseView, Logic Analyzer	LIN Master/Slave, schedule capture	M2 CAN/LIN network bring-up
6	Gateway RTOS	FreeRTOS, CubeIDE, draw.io	Task/Queue/Cache skeleton	Task heartbeat + queue evidence
7	CAN-LIN Routing	Routing Table, cantools, PulseView	MVP-1/MVP-2 routing	M3 Gateway MVP
8	ECU Application	CubeIDE, ST-LINK, CAN/LIN tools	VCU/Motor/Cluster/Door/Light 기능	Demo path end-to-end
9	Fault Handling	Jira, Python, python-can, SavvyCAN	timeout/alive/offline/fail-safe	M4 fault cases PASS
10	Diagnostic Tool	VS Code, Python, PySide6, cantools	GUI, logging, injection	PC Tool observes all health/fault
11	Integration / Test	pytest, CANable, PulseView	test-result.md, KPI data	M5 integration regression PASS
12	Final Release / Demo	GitHub Releases, README, Jira	v1.0 assets, demo script, report	Release 1.0 + final demo

13.2 Milestones

표 13-2. Milestone Gate

Milestone	의미	상세 통과조건
M0 Architecture Freeze	전체 구조와 범위 고정	CAN/LIN topological diagram, ECU 역할, 7 인 담당, MVP 와 제외 범위, BOM/보드 계획이 리뷰 완료
M1 Interface Freeze	CAN/LIN Interface 고정	CAN Matrix, vehicle.dbc, LIN Matrix, LIN Schedule, Signal scaling/enum/timeout/alive 가 PR 승인
M2 CAN/LIN network bring-up	두 bus 단독 동작 확인	CAN raw 송수신과 LIN Schedule/response 를 각각 SavvyCAN/PulseView 로 확인
M3 Gateway MVP	핵심 양방향 Routing 확인	Gear R -> ReverseLamp, DoorOpen -> Cluster 두 시나리오 PASS
M4 Fault handling	장애 감지·복구 확인	VCU timeout, Door 3-cycle offline, Alive error, Motor torque=0, Fault 전파 PASS
M5 Integration	전체 ECU 와 PC Tool 통합	Demo 시나리오, pytest regression, KPI 측정, evidence 정리 PASS
Release 1.0	재현 가능한 최종 배포	main tag, GitHub Release, DBC/Firmware/PC Tool/Docs/Test Result/Setup Guide 포함

Gate 운영

Gate 는 발표용 체크박스가 아니라 다음 단계로 넘어갈 수 있는 증거 기준이다. 통과하지 못하면 Jira 에 blocker 또는 Bug 를 남기고, 해결 Issue 와 새 evidence 를 연결한다.

14. 단계별 개발계획

아래 24 단계는 설계에서 Release 까지의 실행 순서다. 각 단계의 완료조건은 다음 단계의 입력이므로, 산출물만 만들고 검증을 미루지 않는다. 단계 내부의 세부 Task 는 Jira 에서 1-3 일 크기로 쪼개고, 인터페이스를 바꾸는 작업은 반드시 상대 담당자 리뷰를 거친다.

14.1 Phase A - 기준선 만들기(STEP 1-4)

STEP 1. Jira/GitHub 개발환경 구축

항목	내용
목표	업무와 코드의 기준점을 만든다.
세부작업	Jira Project, Epic, Board, Status 를 만들고 GitHub Repository 와 protected branch 를 설정한다. 7 명 권한을 부여하고 README, CONTRIBUTING, 첫 Issue 와 첫 PR 을 만든다.
사용할 Tool	Jira, Git, GitHub, VS Code, Markdown
산출물	Jira Board, Epic 목록, Repository, README.md, CONTRIBUTING.md
완료조건	7 명 모두 Issue 작성 -> branch -> commit -> PR -> review 흐름을 한 번 수행

STEP 2. System Architecture 및 Requirements

항목	내용
목표	프로젝트 범위와 인터페이스를 합의한다.
세부작업	Topology, ECU 역할, MVP, 제외 범위, timeout/alive/health 요구사항을 확정하고 CAN ID 와 LIN PID 후보를 리뷰한다.
사용할 Tool	Jira, draw.io, Markdown + GitHub
산출물	requirements.md, system-architecture.drawio/png, 요구사항 목록
완료조건	M0 승인, 미해결 blocker 0, 담당과 검증 방법이 모든 요구사항에 연결

STEP 3. CAN Matrix 설계(Excel)

항목	내용
목표	CAN Message 와 Signal 의 의미·주기·소유자를 고정한다.
세부작업	6 개 핵심 Message 의 ID, DLC, 주기, Sender/Receiver 와 Signal 의 start bit, length, byte order, scaling, enum, alive/invalid 을 작성한다.

항목	내용
사용할 Tool	Excel, Jira, GitHub
산출물	network/can/CAN_Matrix.xlsx
완료조건	인터페이스 리뷰 완료, 중복 ID 0, 필수 필드 누락 0

STEP 4. CANdb++로 DBC 작성

항목	내용
목표	설계 Matrix 를 실제 CAN Database 로 만든다.
세부작업	CANdb++ 기본 무료 버전에서 VCU, MOTOR, GATEWAY, CLUSTER Node 와 Message, Signal, Value Table, Sender/Receiver 를 정의하고 vehicle.dbc 를 생성한다.
사용할 Tool	CANdb++, Excel, Git
산출물	network/can/vehicle.dbc
완료조건	Excel 과 DBC 가 1:1 로 대응하며 CANdb++ Admin 은 사용하지 않아도 재현 가능

14.2 Phase B - Interface 와 단독 Bus 검증(STEP 5-12)

STEP 5. cantools 로 DBC Validation

항목	내용
목표	DBC 를 PC 와 테스트 코드에서 재현 가능하게 검증한다.
세부작업	dump/parse, ID/DLC, Signal overlap, factor/offset, encode/decode, value table 을 확인하고 fixture 와 검증 로그를 저장한다.
사용할 Tool	cantools, Python, canmatrix, pytest
산출물	DBC validation log, encode/decode fixture, 검증 스크립트
완료조건	parse error 0, encode/decode PASS, overlap 0, 기준값과 invalid 처리 확인

STEP 6. LIN Matrix 설계

항목	내용
목표	LIN Frame 과 Signal 의 의미를 고정한다.
세부작업	Door_Status, Door_Command, Light_Command, Light_Status 의 PID, 주기, Publisher/Subscriber, 데이터와 checksum 응답 규칙을 정의한다.
사용할 Tool	Excel, Jira, Markdown
산출물	network/lin/LIN_Matrix.xlsx

항목	내용
완료조건	Door/Light/Gateway 담당 리뷰 완료, 모든 Frame 의 주기와 응답 주체 확정

STEP 7. LIN Schedule 설계

항목	내용
목표	Gateway LIN Master 의 polling 순서를 재현 가능하게 만든다.
세부작업	0/20/40/60/100 ms Schedule 과 20 ms tick, slot 지연, no response 처리, repeat 조건을 정한다.
사용할 Tool	Excel, draw.io, Jira
산출물	network/lin/LIN_Schedule.xlsx, Schedule diagram
완료조건	slot 충돌 없음, PulseView 측정 기준과 timeout 판정 시점이 명시됨

STEP 8. MCU Peripheral 설계(CubeMX)

항목	내용
목표	보드별 clock 과 통신 peripheral 을 동일 기준으로 만든다.
세부작업	.ioc 에 clock, GPIO, CAN 500 kbps, UART logging, LIN 19.2 kbps, RTOS tick, transceiver enable 을 설정하고 생성 차이를 리뷰한다.
사용할 Tool	STM32CubeMX, Git, Jira
산출물	.ioc, generated init code, peripheral configuration note
완료조건	clean generate/build, 핀맵과 bitrate/baudrate 를 담당자들이 확인

STEP 9. Firmware Bring-up(CubeIDE/ST-LINK/UART)

항목	내용
목표	최소 실행 firmware 를 모든 보드에서 확인한다.
세부작업	Build, flash, boot banner, LED heartbeat, scheduler, CAN/LIN/UART skeleton 을 순서대로 올리고 보드별 로그를 저장한다.
사용할 Tool	STM32CubeIDE, ST-LINK, CoolTerm/Tera Term
산출물	boot log, flash evidence, 보드별 bring-up 기록
완료조건	7 개 역할 보드 또는 할당 환경에서 flash 성공, UART banner 와 heartbeat 확인

STEP 10. UART Logging 표준

항목	내용
목표	모든 ECU 로그를 같은 형식으로 만든다.

항목	내용
세부작업	timestamp, module, level, event, key=value 형식과 log level 을 정하고 ISR 에서는 직접 printf 하지 않으며 LoggerTask 로 전달한다.
사용할 Tool	STM32CubeIDE, UART terminal, Markdown
산출물	coding-convention.md 보강, UART log sample
완료조건	모든 ECU 에 boot/health/fault 공통 log 가 있고 timestamp 로 bus evidence 와 대조 가능

STEP 11. CAN Network 단독 Bring-up(CANable/SavvyCAN)

항목	내용
목표	Gateway 없이 CAN bus 자체를 검증한다.
세부작업	CANable 을 500 kbps 로 설정하고 raw ID, DLC, payload, period, termination, ACK 상태를 확인한다.
사용할 Tool	CANable, SavvyCAN, cantools
산출물	CAN capture, cycle measurement, wiring/termination 기록
완료조건	대표 Message 가 raw 와 DBC decode 에서 일치하고 주기 허용오차 안에 있음

STEP 12. LIN Network 단독 Bring-up(CubeIDE/PulseView)

항목	내용
목표	Master/Slave waveform 과 응답을 검증한다.
세부작업	Schedule, break/sync/PID/data/checksum, 19.2 kbps waveform, 정상 응답과 no response 를 Logic Analyzer 로 캡처한다.
사용할 Tool	STM32CubeIDE, USB Logic Analyzer, PulseView
산출물	LIN capture, timing 측정표, Slave UART/firmware log
완료조건	각 Frame slot 과 timing 의 증거가 있고 no response 재현 가능

14.3 Phase C - Gateway 와 ECU 기능(STEP 13-19)

STEP 13. Gateway FreeRTOS Architecture

항목	내용
목표	Gateway 의 task, queue, cache, health 경계를 고정한다.
세부작업	ISR minimal work, CANRxQueue, CANRxTask, Signal Cache, RoutingTask, LINSchedulerTask, HealthTask, LoggerTask 의 우선순위와 overflow 정책을 구현한다.
사용할 Tool	FreeRTOS, STM32CubeIDE, draw.io, Jira

항목	내용
산출물	gateway-sw-architecture.drawio, task design, queue policy
완료조건	priority/period/queue depth/overflow log 와 task heartbeat 가 확인됨

STEP 14. CAN<->LIN Routing

항목	내용
목표	양방향 Routing MVP 를 구현한다.
세부작업	Signal Cache 와 routing_table.c/.h 를 이용해 Gear R -> ReverseLamp 와 DoorOpen -> Body_Status 를 conversion 후 전달한다.
사용할 Tool	CubeIDE, cantools, SavvyCAN, PulseView
산출물	Routing Table, routing test evidence, MVP capture
완료조건	MVP-1 과 MVP-2 모두 CAN/LIN raw, decoded signal, output 상태로 PASS

STEP 15. VCU Application

항목	내용
목표	입력에서 Vehicle State 와 Torque Request 를 만든다.
세부작업	Ignition, accelerator, brake, gear state machine 을 만들고 TorqueRequest, alive, timeout 상태를 CAN 10 ms 로 전송한다.
사용할 Tool	CubeIDE, CANdb++, SavvyCAN, pytest
산출물	VCU firmware, CAN capture, state/timeout log
완료조건	VCU_Command 10 ms 주기와 Gear/Torque/Brake decode 가 일치

STEP 16. Virtual Motor ECU

항목	내용
목표	실제 모터 없이 Motor ECU 동작을 재현한다.
세부작업	TorqueRequest -> target RPM -> RPM 응답 모델을 만들고 Motor_Status 를 전송한다. VCU offline 이면 torque=0 으로 fail-safe 한다.
사용할 Tool	CubeIDE, Python, cantools, pytest
산출물	Motor firmware/model, response log, timeout test
완료조건	입력 torque 변화에 RPM 이 응답하고 VCU timeout 후 0 으로 수렴

STEP 17. Cluster

항목	내용
목표	차량/Body/Fault 상태를 사람이 확인할 수 있게 표시한다.
세부작업	Speed, RPM, Gear, Brake, Door/Light, Fault 와 ECU online/offline 및 stale 상태를 수신해 표시한다.
사용할 Tool	CubeIDE, CANable, SavvyCAN, PC Diagnostic Tool
산출물	Cluster firmware, display evidence, fault display evidence
완료조건	accelerator, reverse, door open, offline demo 에서 표시가 요구사항과 일치

STEP 18. Door ECU

항목	내용
목표	LIN Slave 로 Door 상태와 명령을 구현한다.
세부작업	Door open/close, lock/unlock, Door_Status/Command 응답을 만들고 응답 중단 fixture 로 offline 을 재현한다.
사용할 Tool	CubeIDE, PulseView, LIN Matrix
산출물	Door firmware, LIN capture, response-stop fixture
완료조건	DoorOpen path 가 동작하고 3-cycle no response 를 재현

STEP 19. Light ECU

항목	내용
목표	LIN Slave 로 Body Lamp 를 구현한다.
세부작업	Head, turn, hazard, reverse, brake lamp 의 command/status 를 만들고 ReverseLamp output 을 확인한다.
사용할 Tool	CubeIDE, PulseView, LIN Schedule
산출물	Light firmware, lamp output evidence, status capture
완료조건	Gear=R Routing 후 ReverseLamp 가 ON 이 되고 Light_Status 로 확인

14.4 Phase D - Fault, Tool, Test, KPI(STEP 20-24)

STEP 20. Fault Handling(CAN timeout, LIN timeout, alive counter, ECU offline)

항목	내용
목표	장애를 감지하고 fail-safe 와 전파를 일관되게 수행한다.

항목	내용
세부작업	VCU 10 ms/30 ms timeout, Door 100 ms/3-cycle offline, alive 0..15 discontinuity, Motor torque=0, Cluster/PC fault 전달과 recovery 를 구현한다.
사용할 Tool	FreeRTOS, Python, SavvyCAN, PulseView, Jira
산출물	fault design, fault capture, test result, recovery log
완료조건	지정 threshold 와 Fault code/action 이 모두 재현되고 복구 조건이 문서화됨

STEP 21. Fault Injection(Python/python-can)

항목	내용
목표	장애를 반복 가능한 입력으로 만든다.
세부작업	CAN frame drop, wrong alive, invalid value, LIN response stop, recovery 를 fixture 와 config 로 실행한다.
사용할 Tool	Python, python-can, cantools, CANable
산출물	fault_injection scripts, config, injection log
완료조건	같은 입력에서 같은 fault 와 detection time 이 나오고 Jira Bug 에 연결 가능

STEP 22. PC Diagnostic Tool(VS Code/Python/python-can/cantools/PySide6)

항목	내용
목표	차량 상태를 관찰·해석·기록·주입한다.
세부작업	CAN Interface -> DBC Decoder -> Signal DB -> Logger -> GUI 구조로 CLI 를 먼저 만들고 PySide6 화면을 추가한다.
사용할 Tool	VS Code, Python, python-can, cantools, PySide6
산출물	tools/diagnostic_gui, setup guide, screens, log files
완료조건	Vehicle Status, ECU Health, Network Status, raw/decode, logging, injection, graph 가 동작

STEP 23. Automated Test(pytest)

항목	내용
목표	정상·Fault·MVP 를 반복 검증한다.
세부작업	test_vcu.py, test_motor.py, test_gateway.py, test_routing.py, test_can_timeout.py, test_lin_timeout.py, test_alive_counter.py 등을 작성한다.
사용할 Tool	pytest, Python, python-can, cantools
산출물	tests/, test-result.md, regression log

항목	내용
완료조건	nominal/fault/MVP test PASS, 실패는 Jira Bug 와 evidence 에 연결

STEP 24. Performance/KPI 측정

항목	내용
목표	감이 아닌 숫자로 Release 품질을 판단한다.
세부작업	CAN cycle/jitter/loss, LIN timing, CAN->LIN latency, timeout, alive fault, queue usage/drop, recovery time 을 샘플링하고 min/max/avg/P95 를 기록한다.
사용할 Tool	SavvyCAN, PulseView, Python, Gateway_Status, pytest
산출물	KPI CSV/graph, performance section, test-result.md
완료조건	각 KPI 에 실제 샘플 수와 숫자 결과가 있고 목표 대비 판정이 명확

14.5 Final Integration/Release

표 14-1. Final Integration/Release

항목	내용
목표	전체 ECU, CAN/LIN Gateway, PC Diagnostic Tool 을 하나의 재현 가능한 데모와 Release package 로 묶는다.
세부작업	MVP 와 최종 Demo 시나리오를 순서대로 실행하고, fault injection 과 pytest regression 을 수행한다. KPI 실제 수치, 배선/bitrate 설정, firmware commit, DBC 버전을 evidence 에 연결한다. main 에 release tag 를 만들고 GitHub Releases 에 산출물을 첨부한다.
사용할 Tool	Jira, Git/GitHub, GitHub Releases, CANable, SavvyCAN, USB Logic Analyzer, PulseView, STM32CubeIDE, ST-LINK, PC Diagnostic Tool, pytest
산출물	v1.0 tag/release, vehicle.dbc, firmware binaries/source, PC Tool, test-result.md, KPI CSV/graph, demo script, setup guide
완료조건	모든 Release DoD 가 PASS 이고 Jira Done 기준을 만족한다. Merge 만 된 작업, 숫자 없는 KPI, 재현되지 않는 데모는 Release 완료로 보지 않는다.

개발 순서의 핵심

Interface 를 먼저 고정하고 단독 Bus 를 검증한 뒤 Gateway Routing 과 ECU Application 을 합친다. 마지막에 Fault 와 PC Tool 을 덧붙이는 것이 아니라, 각 단계에서 evidence 를 남겨 통합 시 원인을 좁힐 수 있게 한다.

15. Jira + Git/GitHub 협업 매뉴얼

7 명이 동시에 firmware, database, test, PC Tool 을 만지면 작업 위치와 완료 의미가 섞이기 쉽다. 이 장에서는 업무 상태는 Jira, 실제 파일과 변경 이력은 Git/GitHub 라는 경계를 고정한다. GitHub Issues 는 사용하지 않고 Jira 를 단일 Task 시스템으로 사용한다.

15.1 Jira 와 Git/GitHub 의 책임 분리

표 15-1. 협업 도구의 책임 경계

영역	담당 기준	반드시 남길 것
Jira	업무, 일정, 담당자, Bug, Test 상태	Issue Key, 목적, 완료조건, 담당자, 우선순위, 의존성, evidence 링크
Git	실제 코드·설정·문서의 변경 이력	branch, commit, diff, tag, local history
GitHub	원격 저장소, PR, review, release, docs 공유	PR, reviewer, approval, merge commit, tag, GitHub Release

단일 기준

Jira 는 무엇을 왜 언제 누가 하는지를 관리하고, Git/GitHub 는 무엇이 어떻게 바뀌었는지를 관리한다. GitHub PR 링크만 남기고 Jira 완료조건을 비워 두지 않는다.

15.2 Jira Epic 구조

표 15-2. Jira Epic 목록

Epic Key	Epic 이름	대표 범위
SYS	System	Architecture, Requirements, BOM, 공통 규칙
CAN	CAN	CAN Matrix, CANdb++, vehicle.dbc, CAN validation
LIN	LIN	LIN Matrix, Schedule, Master/Slave, waveform
GW	Gateway	FreeRTOS, cache, routing, health, fault
VCU	VCU	Input, state machine, torque request, alive
MOTOR	Motor ECU	Virtual motor, RPM model, status, fail-safe
CLUSTER	Cluster ECU	Vehicle/Body/Fault 표시, stale/offline
DOOR	Door ECU	LIN Door status/command, offline fixture

Epic Key	Epic 이름	대표 범위
LIGHT	Light ECU	Lamp command/status, Reverse/Brake/Turn
TOOL	PC Diagnostic Tool	CAN monitor, decode, GUI, logging, injection
TEST	Test & KPI	pytest, integration, fault, performance, release evidence

15.3 Issue Type 와 Board Status

표 15-3. Issue Type

항목	사용 방법	예시
Epic	큰 기능 또는 workstream. 여러 Task/Bug/Test 를 묶는다.	GW, CAN, TEST
Task	1-3 일에 끝낼 수 있는 구체적인 작업.	GW-23 ReverseLamp routing
Bug	재현 조건과 기대/실제 결과가 있는 결함.	LIN timeout 이 3 cycle 전에 발생
Test	검증 절차, fixture, 측정 또는 regression.	TEST-12 CAN timeout

그림 15-1. 권장 Board 상태 흐름

Backlog -> To Do -> In Progress -> Code Review -> Test -> Done
Merge != Done PR 이 merge 되면 Code Review 단계의 작업이 통합 branch 에 들어온 것이다. 실제 보드 상태는 HW 또는 자동화 Test PASS, 완료조건 확인, evidence 연결까지 끝난 뒤 Done 으로 전환한다.

15.4 좋은 Task 의 크기와 표현

Task 는 한 사람이 1-3 일 안에 완료할 수 있고, 결과를 Build·Capture·Log·Test 중 하나로 증명할 수 있어야 한다. Gateway 개발처럼 범위가 넓은 문장을 하나의 Task 로 만들면 담당자와 완료조건이 흐려진다.

표 15-4. Task 작성 예시

구분	예시	문제 또는 장점
나쁜 예	Gateway 개발	범위, 입력, 완료조건, 검증 방법이 없다.
좋은 예	GW-01 CAN init	CAN bitrate 와 init 완료 log 를 증명할 수 있다.
좋은 예	GW-02 CAN RX interrupt	ISR 동작과 최소 처리 원칙을 검증할 수 있다.
좋은 예	GW-03 CAN RX queue	Queue depth, overflow policy, drop counter 를 정의할 수 있다.

구분	예시	문제 또는 장점
좋은 예	GW-04 message decode	vehicle.dbc 의 특정 Message/Signal decode 를 확인할 수 있다.
좋은 예	GW-05 LIN master init	LIN 19.2 kbps Master 초기화를 단독 검증할 수 있다.
좋은 예	GW-06 scheduler	0/20/40/60 ms slot 과 repeat 를 capture 할 수 있다.
좋은 예	GW-07 routing manager	Routing Table 과 하나의 path 를 end-to-end 검증할 수 있다.
좋은 예	GW-08 CAN timeout	30 ms threshold 와 VCU_TIMEOUT action 을 시험할 수 있다.
좋은 예	GW-09 LIN timeout	3-cycle no response 와 DOOR_ECU_OFFLINE 을 재현할 수 있다.

15.5 Jira Task Template - GW-23 예시

GW-23 [Gateway] VCU Gear -> LIN ReverseLamp Routing 구현

목적: VCU_Command 의 Gear signal 이 R 일 때 Gateway 가 LIN Light_Command 의 ReverseLamp 를 1 로 만든다.

입력: CAN 0x100, Gear 3 bit, vehicle.dbc 정의, CAN 500 kbps.

출력: LIN Light_Command PID 0x12, ReverseLamp=1, Light_Status 응답.

완료조건: Gear R 입력을 CANable/SavvyCAN 에서 발생시키고, Gateway decode 와 routing log, PulseView LIN frame, Light output 또는 status, Cluster/PC decode 를 함께 확인한다.

표 15-5. Jira Task 필드

필드	작성 기준
Purpose	왜 필요한 기능인지 한 문장으로 쓴다.
Input	Network, Message/PID, Signal, 값 범위, 주기를 쓴다.
Output	Destination Network/Message/Signal 과 변환 규칙을 쓴다.
Acceptance Criteria	raw capture, decoded value, output state, timeout/recovery 까지 측정 가능한 문장으로 쓴다.
Dependencies	CAN Matrix, vehicle.dbc, LIN Schedule, 상대 담당자 Issue 를 링크한다.
Evidence	PR, capture, UART log, pytest result, 사진 또는 KPI 파일을 연결한다.

15.6 Definition of Done

공통 DoD 는 기능의 종류와 관계없이 지키는 최소선이다. 통신 기능은 코드만으로 완료할 수 없으며, 실제 보드와 bus evidence 가 필요하다.

표 15-6. 공통 Definition of Done

공통 확인항목	Done 기준
Build	warning 정책을 포함해 clean build PASS. 생성물과 commit 을 식별할 수 있다.
Flash / Boot	대상 MCU 에 flash 하고 boot banner, heartbeat, health log 를 확인한다.
Bus evidence	CANable/DBC decode 또는 PulseView waveform 으로 실제 송수신을 확인한다.
기능 Test	Jira 완료조건에 정상·경계·fault 케이스를 실행하고 결과를 기록한다.
Review	최소 1 명 승인. CAN/LIN interface 변경은 상대 담당자 또는 System/Gateway 담당자가 리뷰한다.
문서	변경된 Matrix, DBC, Schedule, setup/test 문서를 함께 갱신한다.
Jira	Issue 상태, PR 링크, evidence 링크, 알려진 제한사항을 갱신한다.

표 15-7. 기능별 추가 DoD

기능 유형	추가 DoD
CAN Message	CAN Matrix 와 vehicle.dbc 가 일치하고 ID/DLC/Signal encode/decode 및 주기 측정 PASS
LIN Frame	LIN Matrix/Schedule 와 PID/data/checksum 응답이 일치하고 PulseView timing PASS
Routing	source/destination raw frame, decoded value, conversion, output 상태를 양방향으로 확인
Fault	fault code, threshold, fail-safe action, propagation, recovery, 로그를 모두 확인
PC Tool	실제 CAN interface 에서 monitor/decode/logging 및 최소 한 개 injection 을 수행

15.7 Git Branch 전략

코드 15-1. Branch 전략과 이름

```

main      - Release 전용. 보호 branch, 직접 push 금지
develop   - 팀 통합 branch. feature/fix/test 결과를 모으는 기준
feature/* - 새 기능 구현
fix/*     - Bug 수정
test/*    - Test fixture, 자동화, 측정 및 evidence 보장

```

예시
feature/GW-23-reverse-lamp-routing
fix/GW-31-lin-timeout
test/TEST-12-can-timeout
feature/VCU-11-torque-request

코드 15-2. 개발 시작 명령

```

git checkout develop
git pull
git checkout -b feature/GW-23-reverse-lamp-routing

```

Branch 규칙

모든 branch 이름에 Jira Key 를 넣는다. 기능 branch 는 develop 에서 만들고, main 은 Release 때만 develop 에서 승격한다. branch 가 오래 살아남지 않도록 작은 PR 로 자주 통합한다.

15.8 Commit 규칙

Commit 은 의미 있는 작은 단위로 남긴다. 한 commit 에서 unrelated formatting, generated file, 기능 변경을 뒤섞지 않는다.

표 15-8. Commit 메시지

구분	예시
좋은 예	GW-23 add CAN gear signal decoder
좋은 예	GW-23 implement reverse lamp routing
좋은 예	GW-23 add LIN light command encoder
좋은 예	TEST-12 add CAN timeout fixture
좋은 예	CAN-08 update vehicle.dbc for Gear enum
나쁜 예	수정 / 테스트 / 최종 / 진짜최종 / fix / 수정 2

15.9 Pull Request 규칙

PR 은 코드 묶음을 올리는 창구이자 Jira 완료조건을 검토하는 문서다. 제목만 보고도 Issue 와 기능을 알 수 있어야 한다.

코드 15-3. PR Template

제목: [GW-23] Implement Reverse Lamp Routing

Jira Issue

- GW-23 링크

구현 내용

- CAN VCU_Command Gear decode
- Routing Table conversion
- LIN Light_Command ReverseLamp encode

테스트 환경

- board: STM32F446RE
- CAN: 500 kbps
- LIN: 19.2 kbps
- tools: CANable, SavvyCAN, PulseView

Test result

- build: PASS
- CAN raw/decode: PASS
- LIN capture: PASS
- hardware output: PASS

Evidence

- capture, UART log, pytest result 링크

리뷰 요청사항

- conversion 과 timeout 경계 확인

Known limitations

- 실제 lamp 대신 GPIO output 사용

표 15-9. PR 처리 순서

PR 단계	기준
Open	Jira Key, 목적, 변경 파일, 테스트 환경, 결과, evidence 를 작성
Review	최소 1명 Approve. Interface PR 은 상대 담당자가 확인
Merge	feature -> develop. 충돌과 CI/build 확인 후 merge
Test	실제 HW 또는 자동화 Test 를 수행하고 Jira 에 결과 기록
Done	완료조건, evidence, 문서, known limitation 확인 후 Jira Done

리뷰의 기준

리뷰어는 문법만 보지 않는다. signal bit/scale, 주기, ISR 처리량, queue overflow, timeout threshold, fail-safe, 실제 증거까지 확인한다.

15.10 Bug 처리 흐름

그림 15-2. Bug workflow

```

Jira Bug 생성
-> 재현 조건과 기대/실제 결과 기록
-> fix branch 생성
-> 원인 분석 및 수정
-> PR / review
-> develop merge
-> HW 또는 integration Test
-> PASS: 원래 Bug 에 evidence 연결 후 Done
-> FAIL: 결과를 갱신하고 linked Bug 또는 후속 Issue 생성

```

Bug 에는 어느 commit, 보드, bus 설정, 입력 frame, 기대값, 실제값, 로그와 capture 를 사용했는지 남긴다.

'안 됨'만으로는 재현 가능한 Bug 가 아니다.

15.11 전체 Jira/GitHub Workflow

그림 15-3. Jira 와 GitHub 의 전체 연결 흐름

```

Jira Task
-> To Do
-> In Progress
-> GitHub feature branch
-> Commit with Jira Key
-> Pull Request
-> Jira Code Review
-> GitHub Review / Approve
-> Merge to develop
-> Jira Test
-> HW / automated test
  -> PASS: Done
  -> FAIL: Bug -> fix branch

```

15.12 1 주 Sprint 운영

표 15-10. 1 주 Sprint 운영

시점	운영 내용	남길 것
월요일 - Planning	지난주 Done 과 미완료를 확인하고 blocker 를 정리한다. 이번 주 To Do, 담당자, 예상 evidence 를 배정한다.	Sprint 목표, Jira assignment, dependency
화-금 - Development	각자 Issue 단위로 개발한다. 인터페이스 변경은 즉시 Jira 와 Matrix/DBC/Schedule 에 반영하고 리뷰를 요청한다.	branch, commit, PR, log/capture
주말 - Integration 권장	가능하면 Gateway 와 ECU 를 연결해 전체 path 와 fault 를 한 번 실행한다. 불가능하면 다음 Planning 전에 integration slot 을 잡는다.	integration result, Bug, KPI sample
Sprint 종료	Done 기준을 다시 확인한다. merge 만 된 작업은 Done 으로 올리지 않고 Test 또는 In Progress 에 둔다.	Sprint result, carry-over reason

회의 부담

매일 긴 Daily Scrum 까지는 필요하지 않다. 월요일 Planning 과 비동기 Jira 업데이트, 필요할 때의 짧은 interface review 만으로도 7 인 프로젝트의 흐름을 유지할 수 있다.

15.13 팀 공통 개발규칙 7 개

- Jira Task 없이 코딩하지 않는다.
- main 에 직접 push 하지 않는다.
- branch, commit, PR 제목에 Jira Key 를 넣는다.
- 모든 PR 은 최소 1 명에게 review/Approve 를 받는다.
- Merge 는 Done 이 아니다. HW 또는 automated Test PASS 후 Done 이다.
- CAN/LIN interface 변경은 반드시 상대 담당자 또는 System/Gateway 담당자가 리뷰한다.
- 문제는 채팅에만 남기지 않고 Jira Bug 와 evidence 로 등록한다.

15.14 Interface 변경 리뷰 원칙

표 15-11. Interface 변경 리뷰

변경 대상	반드시 함께 리뷰할 사람	영향 파일/검증
CAN ID, DLC, Signal bit/scale/enum	CAN 담당 + 송신/수신 ECU 담당 + Gateway	CAN_Matrix.xlsx, vehicle.dbc, cantools encode/decode, raw capture
LIN PID, Frame, period, Schedule	LIN common + Door/Light + Gateway	LIN_Matrix.xlsx, LIN_Schedule.xlsx, PulseView capture
Timeout, Alive, Fault code	Gateway + 해당 ECU + Cluster/PC Tool	requirements, fault table, Fault_Status, recovery test

변경 대상	반드시 함께 리뷰할 사람	영향 파일/검증
Routing conversion	System/Gateway + source/destination ECU	routing_table.c/.h, MVP capture, pytest

16. Sprint 예시와 최종 치트시트

첫 세 Sprint 는 문서와 Interface 를 먼저 고정하고, 세 번째 Sprint 부터 ECU 별 parallel bring-up 으로 전환하는 예시다. 실제 Jira 날짜와 담당자는 팀의 시작일에 맞춰 입력하되, Issue Key 와 완료조건 형식은 유지한다.

16.1 Sprint 1 - 기준선과 개발환경

표 16-1. Sprint 1 Jira Task 예시

Issue	Task	세부 내용	산출물 / 완료조건
SYS-01	Architecture	최종 CAN/LIN topology 와 Gateway 경계를 그린다.	system-architecture.drawio/png, 팀 리뷰 승인
SYS-02	Requirements	MVP, timeout, alive, health, fail-safe 요구사항을 쓴다.	requirements.md, 요구사항별 검증 방법
SYS-03	ECU function	VCU/Motor/Gateway/Cluster/Door/Light/PC Tool 역할을 확정한다.	ECU 역할표, 담당 연결
SYS-04	Topology	CAN 500 kbps, LIN 19.2 kbps, CANable 위치와 Gateway Master 를 확정한다.	Network diagram, wiring note
SYS-05	BOM	보드, transceiver, CANable, Logic Analyzer, ST-LINK, 배선을 정리한다.	BOM 과 보유/대여 상태
SYS-06	Jira setup	Project, Epic, Issue Type, Board Status, 권한과 Sprint 를 만든다.	Jira Board, Epic 목록
SYS-07	GitHub setup	Repository, develop, protected main, PR rule 을 만든다.	Repository, branch protection, PR template
SYS-08	Coding convention	C naming, log, commit, branch, review 규칙을 작성한다.	coding-convention.md, CONTRIBUTING.md
SYS-09	Dev environment	CubeMX/CubeIDE/Python/cantools/pytest/terminal 환경을 확인한다.	setup checklist, 첫 Build/pytest PASS

Sprint 1 Gate

M0 후보를 승인할 때는 예쁜 diagram 보다 범위, 담당, MVP, 제외 범위, 검증 방법이 실제로 연결되어 있는지를 본다.

16.2 Sprint 2 - CAN/LIN Interface Freeze 준비

표 16-2. Sprint 2 Jira Task 예시

Issue	Task	세부 내용	산출물 / 완료조건
-------	------	-------	------------

Issue	Task	세부 내용	산출물 / 완료조건
CAN-01	Message Matrix	VCU_Command, Motor_Status 등 ID/DLC/주기를 정리한다.	CAN_Matrix.xlsx 초안
CAN-02	Signal Matrix	TorqueRequest, Gear, Brake, AliveCounter 와 scaling/enum 을 정리한다.	Signal field complete
CAN-03	Review	송수신 ECU 와 Gateway 가 CAN Matrix 를 함께 검토한다.	중복 ID/overlap/receiver 이슈 0
CAN-04	CANdb++ Node	VCU, MOTOR, GATEWAY, CLUSTER Node 를 만든다.	CANdb++ database
CAN-05	CANdb++ Messages	Message ID, DLC, cycle, Sender/Receiver 를 입력한다.	Message definition
CAN-06	CANdb++ Signals	Start bit, length, byte order, factor, offset, range, unit, enum 을 입력한다.	Signal/Value Table definition
CAN-07	vehicle.dbc	CANdb++에서 DBC 를 생성하고 Matrix 와 diff 한다.	network/can/vehicle.dbc
CAN-08	cantools validation	dump, parse, encode/decode, overlap, value table 을 검증한다.	validation log, PASS
LIN-01	Frame Matrix	Door/Light Frame PID 와 Publisher/Subscriber 를 정리한다.	LIN_Matrix.xlsx
LIN-02	Signal Matrix	DoorOpen, Lock, ReverseLamp, BrakeLamp 등을 정의한다.	LIN signal field complete
LIN-03	Schedule	0/20/40/60/100 ms slot 과 repeat 를 정한다.	LIN_Schedule.xlsx
GW-01	Gateway SW architecture	Cache, routing, health, logger 경계를 그린다.	gateway-sw-architecture.drawio
GW-02	RTOS task design	CANRxTask/RoutingTask/LINSchedulerTask/Health Task/LoggerTask 를 설계한다.	priority, queue, period, overflow policy
GW-03	routing architecture	source -> conversion -> destination 모델과 routing_table 파일 구조를 정한다.	routing_table.c/.h skeleton

Sprint 2 Gate

M1 Interface Freeze 는 Excel 만 승인하는 것이 아니다. Excel -> CANdb++ -> vehicle.dbc -> cantools 와 LIN Matrix -> Schedule 까지 실제 검증 가능한 파일이 함께 승인되어야 한다.

16.3 Sprint 3 - ECU 별 parallel bring-up

표 16-3. Sprint 3 parallel work

Workstream	병렬 작업 예시	공유 Interface / 종료 증거
VCU	VCU CAN init, input mock, state machine, VCU_Command 10 ms, alive	CAN_Matrix/vehicle.dbc, SavvyCAN capture, UART state log
Motor	Virtual Motor model, target RPM, response filter, VCU timeout torque=0	VCU_Command/Motor_Status, RPM graph, timeout test

Workstream	병렬 작업 예시	공유 Interface / 종료 증거
Gateway	CAN RX/TX, LIN Master, 20 ms scheduler, cache, MVP routing	Gateway task log, CAN/LIN capture, MVP-1/2
Cluster	Vehicle/Body/Fault display 와 stale/offline 표시	Vehicle_Status/Body_Status/Fault_Status decode, 화면 evidence
Door	LIN Slave Door_Status/Command, open/close/lock/unlock, no response fixture	LIN capture, DoorOpen, 3-cycle offline
Light	LIN Slave Light_Command/Status, head/turn/hazard/reverse/brake	PulseView, ReverseLamp output/status
PC/Test	python-can monitor, cantools decode, logger, pytest, fault injection	test-result, raw/decode log, reproducible script

parallel 작업의 전제는 Interface 가 이미 M1 기준으로 리뷰되었다는 것이다. 구현 중 Message/PID/Signal 정의가 바뀌면 해당 Jira Issue 를 멈추고 Interface Change 흐름으로 되돌아간다.

최종 한 장 치트시트

가장 중요한 개발 흐름

Jira Issue -> Git Branch -> Coding -> Commit -> PR -> Review -> Merge -> HW Test -> Done

Tool Chain

표 16-4. 최종 Tool Chain

대상	표준 흐름	핵심 확인
CAN	Excel -> CANdb++ -> vehicle.dbc -> cantools -> CANable/SavvyCAN -> ECU/PC Tool	Matrix 와 DBC 일치, raw/decode/주기
LIN	Excel -> LIN Matrix/Schedule -> CubeIDE -> Logic Analyzer/PulseView	Master schedule, PID/data/checksum, timing
Firmware	CubeMX -> CubeIDE -> ST-LINK -> CAN/UART/LIN 검증	build/flash/boot/bus evidence
Test	Python -> python-can -> cantools -> pytest	nominal/fault/regression 과 실제 결과

Jira 상태 전환

코드 16-1. Jira 상태와 Done

Backlog -> To Do -> In Progress -> Code Review -> Test -> Done

Done 조건:
Build PASS + Flash/Boot + HW/automated Test PASS
+ PR review + evidence + 문서/Jira 갱신
Merge 만으로 Done 전환 금지

Branch / Commit / PR naming

표 16-5. Naming 치트시트

대상	규칙	예시
Branch	type/JIRAKEY-short-description	feature/GW-23-reverse-lamp-routing
Fix branch	fix/JIRAKEY-short-description	fix/GW-31-lin-timeout
Test branch	test/JIRAKEY-short-description	test/TEST-12-can-timeout
Commit	Jira Key + 동사 + 변경 내용	GW-23 implement reverse lamp routing
PR	[Jira Key] + English feature summary	[GW-23] Implement Reverse Lamp Routing

Network specs

표 16-6. Network specs

항목	기준
CAN	500 kbps / VCU, Motor, Gateway, Cluster / PC 는 CANable 로 연결
LIN	19.2 kbps / Gateway Master / Door, Light Slave
CAN 핵심 주기	VCU_Command 10 ms, Motor_Status 10 ms, Vehicle_Status 20 ms, Body_Status 100 ms, Gateway_Status 100 ms, Fault_Status event
LIN 핵심 주기	Door_Status 100 ms, Door_Command 100 ms, Light_Command 50 ms, Light_Status 100 ms
Fault 기준	VCU 30 ms timeout, Door 3-cycle no response, Alive 0..15 불연속

MVP 와 Demo 핵심

표 16-7. MVP/Demo 치트시트

순서	입력 -> 처리 -> 출력	PASS evidence
MVP-1	VCU Gear=R -> CAN -> Gateway decode/routing -> LIN -> Light Reverse Lamp ON	0x100 raw/DBC, LIN PID 0x12 capture, lamp/status
MVP-2	DoorOpen -> LIN -> Gateway -> CAN Body_Status -> Cluster DOOR OPEN	LIN response, CAN 0x200 decode, Cluster 표시
Fault	VCU 또는 Door 통신 중단 -> timeout/offline -> fail-safe/Fault_Status -> Cluster/PC	threshold time, Fault code, recovery log

Release DoD

- M0-M5 Gate 와 Release 1.0 통과조건을 Jira evidence 로 확인한다.
- vehicle.dbc, LIN Matrix/Schedule, firmware, PC Tool, tests, KPI, setup guide 가 같은 Release tag 에서 재현된다.
- 최종 Demo 7 개 시나리오를 실제 보드와 PC Tool 로 수행한다.
- KPI 는 목표가 아닌 실제 숫자와 샘플 수, 측정 도구, 판정으로 기록한다.
- main 직접 push, Jira 없는 변경, review 없는 merge, HW Test 없는 Done 을 허용하지 않는다.

A. 최종 Demo 시나리오와 체크리스트

최종 Demo 는 기능을 나열하는 자리가 아니라, 요구사항·Interface·Routing·Fault·PC Tool·증거가 하나의 흐름으로 연결되었음을 보여주는 통합 시험이다. 각 시나리오는 사전조건, 입력, 기대 결과, 기록 도구를 갖고 실행한다.

A.1 최종 Demo 시나리오

표 A-1. 최종 Demo 시나리오

순서	시나리오	입력 / 동작	기대 결과	기록할 evidence
1	Ignition ON / ECU online	VCU ignition on. CAN/LIN Gateway 와 모든 ECU boot.	각 ECU health online, Gateway LIN Master polling 시작, PC Tool boot/health 표시.	UART boot log, Gateway_Status, PC ECU Health
2	Accelerator -> Motor RPM	VCU accelerator 입력 -> TorqueRequest 전송.	Motor 가 target RPM 을 계산하고 RPM 이 응답. Cluster 에 speed/RPM 표시.	CAN raw/decode, RPM graph, Cluster 화면
3	Gear R -> Reverse Lamp	VCU Gear=R -> CAN 0x100 -> Gateway routing -> LIN Light_Command.	Light ECU ReverseLamp ON, Light_Status 에 상태 반영.	SavvyCAN 0x100, PulseView PID 0x12, lamp/status
4	DoorOpen -> Cluster	Door ECU DoorOpen 상태를 LIN 으로 응답.	Gateway 가 Body_Status 를 CAN 으로 전송하고 Cluster 가 DOOR OPEN 표시.	LIN capture, CAN 0x200 decode, Cluster 화면
5	Door ECU 통신중단	Door ECU 응답을 멈춘다.	3-cycle no response -> DOOR_ECU_OFFLINE. Fault CAN 으로 Cluster/PC 에 전달.	PulseView no response, Fault_Status, PC offline
6	VCU CAN 끊김	VCU_Command 전송을 중단한다.	30 ms -> VCU_TIMEOUT. Motor torque=0 fail-safe. Cluster/PC fault 표시.	CAN silence timing, Motor torque, Fault/UART log
7	Recovery	Door/VCU 통신을 정상 복구하고 valid/alive 를 보낸다.	fault clear 조건 충족 후 정상 online/routing 으로 복구.	recovery timing, health transition, KPI

A.2 실행 전 체크리스트

- 모든 보드 전원, GND, CAN/LIN transceiver, termination, ST-LINK 와 USB 연결을 확인한다.
- CANable 과 SavvyCAN 을 CAN 500 kbps 로, LIN 측정 장비를 19.2 kbps waveform 확인용으로 준비한다.

- 실행 commit/tag, vehicle.dbc 버전, LIN Matrix/Schedule 버전을 기록한다.
- PC Diagnostic Tool 의 CAN Interface, DBC path, logging directory, timestamp 기준을 확인한다.
- 각 시나리오 전후에 CAN capture, PulseView capture, UART log, 화면 또는 GPIO 상태를 저장한다.
- Fault Injection 을 사용할 경우 입력 설정 파일과 실행 시각을 함께 저장한다.

A.3 Release package 체크리스트

표 A-2. Release package

구성	포함 파일/증거	확인
Documents	README.md, requirements.md, architecture diagrams, coding-convention.md, test-plan.md, test-result.md	☐
CAN	CAN_Matrix.xlsx, vehicle.dbc, cantools validation log, CAN capture	☐
LIN	LIN_Matrix.xlsx, LIN_Schedule.xlsx, Master/Slave source, PulseView capture	☐
Firmware	gateway/vcu/motor/cluster/door/light source, .ioc, build information, binaries	☐
PC Tool	diagnostic_gui source, setup guide, DBC path/config, sample logs	☐
Tests	tests/can, tests/lin, tests/gateway, tests/integration, pytest result, fault injection	☐
KPI	cycle/jitter/loss/latency/timeout/alive/queue/recovery actual numbers	☐
GitHub Release	main tag v1.0, release notes, artifact links, known limitations	☐

A.4 용어 빠른 참조

표 A-3. 용어

용어	의미
DBC	CAN Message 와 Signal 의 ID, bit layout, scaling, enum, 송수신 정보를 담은 CAN Database
CANdb++	Node/Message/Signal 을 편집해 vehicle.dbc 를 만드는 Database 편집 도구
Signal Cache	Gateway 가 최근 수신 Signal 과 timestamp/valid/alive 상태를 보관하는 영역
Routing Table	source Network/Message/Signal 을 conversion 후 destination 으로 연결하는 선언적 규칙
LIN Master	Schedule 에 따라 Header 를 송신하고 Slave Response 를 polling 하는 Gateway
Alive Counter	0..15 를 순환하며 프레임 연속성과 송신 ECU 상태를 확인하는 4 bit counter
Fail-safe	VCU offline 등 위험 상태에서 Motor torque=0 처럼 안전한 출력으로 전환하는 동작
MVP	최소 동작 제품. 이 프로젝트에서는 Gear R/ReverseLamp 와 DoorOpen/Cluster 두 경로
Gate	다음 단계로 넘어갈 수 있는 증거 기반 통과조건
DoD	Issue 를 Jira Done 으로 바꾸기 위한 공통·기능별 완료 기준

마지막 확인

이 매뉴얼의 예시 ID, bit layout, 시간 threshold 는 팀이 실제 보드와 요구사항을 검토하며 확정한다. 단, 확정 과정 자체는 반드시 Jira Issue -> Excel/Matrix -> CANdb++/DBC 또는 LIN Schedule -> 코드 -> Test -> Release evidence 의 추적 흐름을 따른다.